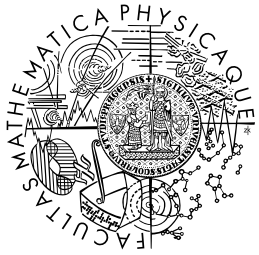

FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

MASTER THESIS

Bc. František Dostál

Fitness and novelty in evolutionary reinforcement learning

Department of Theoretical Computer Science and Mathematical Logic

Supervisor of the master thesis: Mgr. Roman Neruda, CSc.
Study programme: Informatics
Study branch: Artificial Intelligence

Prague 2026

I declare that I carried out this master thesis independently, and only with the cited sources, literature and other professional sources. It has not been used to obtain another or the same degree.

I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ....

Author's signature

I declare that the following generative AI tool was used in the preparation of the submitted thesis in the following ways: The models I used were ChatGPT-5.3-mini and ChatGPT 5.5(https://chat.openai.com/) during the period from April 2026 to July 2026. Their use was limited to specific tasks in the writing process. Namely the search for relevant related literature, stylistic feedback, grammar correction, brainstorming and code snippet generation. I have thoroughly reviewed all AI-generated content to ensure its accuracy and correctness before applying it in the thesis. I take full responsibility for the information presented in the thesis and guarantee its reliability. The use of AI tools was conducted in compliance with ethical standards and academic integrity, ensuring that the work remains original and authentic.

I would like to thank my supervisor Mgr. Roman Neruda, CSc. for patience and guidance. I would also like to thank my friend Martin Mečiar for moral support when it was most needed.

Název práce: Fitness a novelty v evolučním zpětnovazebném učení

Autor: Bc. František Dostál

Katedra: Katedra teoretické informatiky a matematické logiky

Vedoucí práce: Mgr. Roman Neruda, CSc., Katedra teoretické informatiky a matematické logiky

Abstrakt: Tato práce zkoumá využití novelty v evolučních algoritmech při řešení úloh zpětnovazebního učení. Porovnává selekci založenou na novelty se selekcí založenou na fitness, jejich variantami využívajícími archivy a dynamickými kombinacemi novelty a fitness. Porovnání probíhá na prostředích CartPole a Lunar Lander z knihovny Gymnasium. Pro každé prostředí definujeme interpretovatelnou behaviorální charakteristiku a analyzujeme vliv jednotlivých metod práce s objektivní funkcí na dosaženou fitness a diverzitu populace. Výsledky ukazují, že metody založené na fitness představují nejsilnější výchozí přístup z hlediska dosaženého výkonu. Využití novelty zvyšuje diverzitu populace, avšak její přínos závisí na dynamice konkrétní úlohy. Práce dále diskutuje citlivost výsledků algoritmů založených na novelty na volbu behaviorálního prostoru a vzájemné interakce mezi prostředím a evolučním algoritmem.

Klíčová slova: evoluce; novelty; fitness; behaviorální prostor

Title: Fitness and novelty in evolutionary reinforcement learning

Author: Bc. František Dostál

Department: Department of Theoretical Computer Science and Mathematical Logic

Supervisor: Mgr. Roman Neruda, CSc., Department of Theoretical Computer Science and Mathematical Logic

Abstract: This thesis investigates the use of novelty in evolutionary algorithms on reinforcement learning tasks. We compare novelty with fitness based selection, their archival variants and dynamic combinations of novelty and fitness on Gymnasium environments CartPole and Lunar Lander. For each environment we define an interpretable behavioural characteristic and examine how the choice of objective handling method affects the resulting fitness and population diversity. The results show the fitness-based methods achieve best performance under the experiment conditions. The novelty increases population diversity but its benefit depends on specific task dynamics. The thesis also discusses sensitivity of novelty search to the choice of behavioural space and interactions between the environment and the evolutionary algorithm.

Keywords: evolution;novelty; fitness; behavioural space

Contents

Introduction

Evolutionary algorithms are a class of nature based optimisation algorithms predicated on the idea "survival of the fittest" using the optimisation objective as "fitness", guiding the algorithm's search. However, for complex objectives direct use of fitness might trap the algorithm in a false local optimum, unable to reach true global optimum. To solve this problem a variety of approaches have been suggested such as archiving, where variety of individuals is kept by an archive. Another approach is novelty search where fitness is replaced by *novelty*, a difference in behaviour from the rest of the population. In particular, novelty has been shown to work in constrained reinforcement learning tasks such as mazes. The notion of novelty for reinforcement learning has been even generalised outside of the evolutionary algorithms Aubret et al. [2023]. On the other hand there is a number of tasks where combination with fitness might enhance the novelty search, suggesting local competition like MAP-Elites [Mouret and Clune, 2015] or Dominated Novelty Search [Bahlous-Boldi et al., 2025], or other k-neighbours algorithm. These techniques however tend to be computationally rather expensive.

The goal of this thesis is to investigate applications of novelty in evolutionary reinforcement learning and to compare them with direct fitness-based optimisation. We focus on two Gymnasium environments, CartPole and Lunar Lander, for which we design simple interpretable behavioural descriptors. Using evolutionary strategies and differential evolution, we compare fitness, novelty, their archive-based variants, and dynamic combinations of fitness and novelty. The thesis evaluates whether novelty-based objectives improve final task performance, increase behavioural diversity, or exhibit task-dependent limitations caused by the choice of behavioural space and evolutionary algorithm.

In the course of this thesis, we will characterise the two reinforcement learning environments, CartPole and Lunar Lander, suggest an interpretable behavioural representation for them and apply evolutionary algorithm techniques. In the chapter 1 we will introduce terminology pertaining to agents we will be optimising. In chapter 2 we will introduce the aforementioned reinforcement learning tasks and detail the optimising algorithms in chapter 3. In chapter 4 we will review background of the experiments and finally in chapter 5 we will discuss the results of our experiments.

1. Introduction to Artificial Agents

To understand the reasoning behind this work we first examine the fundamental subject of our efforts - the artificial agents.
We will look at their structure, guiding principles, practical implementation and their application.

1.1 Agents and environment

All agents exist in environments that provide context to their agency. An agent receives information about the **environment** it find itself in (i.e. **percepts**) through **sensors** and acts on the environment through **actuators**. Both actuators and sensors are usually among the defining components of the agent. Another defining component of an agent would be setup of its internal decision-making mechanism.

Definition 1 (Russell and Norvig [2021])**.** *Let E be an environment where agent G is located. G is equipped with actuators A and sensors P and so we can identify $obss_P(E)$, a space of all possible sequences of percepts available to G within E and $A(E)$ space of available actions in E. Then function $G : obss_P(E) \to A(E)$ is called an agent function, defining behaviour of the agent G in environment E.*

We illustrate relationship of these concepts in the diagram 1.1. Note that provided definition of agent policy is characteristic to deterministic agents. For agents in stochastic environments we need to reframe the agent function as probabilistic policy. We will discuss this formulation further in section 2.1.

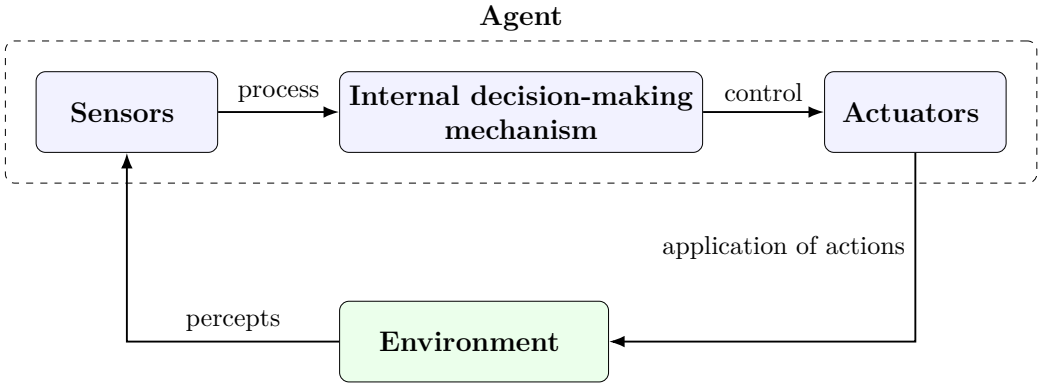

Figure 1.1: Agent and environment relationship

Because we want our agents to solve some tasks in their respective environments with certain efficiency meaning we want them to be intelligent, we need to define a notion of rationality that can help us better capture what we mean by intelligence. First however we need to measure how useful or detrimental a situation can be for our agent.

Definition 2 ([Russell and Norvig, 2021])**.** *Be that E is an environment and $seq(E)$ is a space of all possible sequences of states within E. Then function $f : seq(E) \to \boldsymbol{R}$ is a performance measure of some agent operating in E.*

Definition 3 (Russell and Norvig [2021])**.** *For each possible percept sequence, a rational agent should select an action that is expected to maximise its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.*

We should note that in the real-world applications the rationality of an agents can often be limited by their capacity to store the evidence gathered from percepts, by the lack of their built-in knowledge or the computational capacity to estimate action sequence that can be expected to achieve global maximum of the agent's performance measure.
This is often characterised as a capacity to form *belief*. With that we expand on the notion of agent policy In this thesis we are more concerned with the properties of the evolutionary algorithms rather than evolution of complex agents. Therefore we consider only *reflexive agents*.

Definition 4 ([Russell and Norvig, 2021])**.** *An agent is called simple reflex agent if it takes into account only the current percepts, never keeping any history or state information during its run.*

1.2 Neural Network Agent

The concept of neural networks (NN) comes from an area of machine learning called deep learning that was conceived to attempt to model biological neuron activity. Since it has become one of the most successful branches of machine learning (Russell and Norvig [2021]). The neural networks consist of neurons organised in layers.

Definition 5. *Let $W \in R^{mxn}$ and $b \in R^n$ and let $f : R^n \to R$.Then we call W the weights of a neuron layer with n neurons, b the bias of the layer and f the activation function of the layer when given an input vector $x \in R^m$ we calculate the output of the layer as $f(W^T x + b)$. Two layers have directional connection when the output of first layer is used as input of the subsequent layer.*

The purpose of the activation function is to introduce exploitable non-linearity into the neural system. We recognise several basic activation function.

- sigmoid
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$
- tanh
$$\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$
- ReLU
$$ReLU(x) = max(0, x)$$

Definition 6 (Russell and Norvig [2021])**.** *A neural network where the connections between layers of neural network form an acyclic graph is called a feed-forward neural network.*

Because the feed-forward NN doesn't hold any state or holds onto any previously acquired results and only produces output for a given input, they are ideal for implementation of simple reflex agents. In this thesis we use feed-forward networks exclusively, so any further mention of neural network should be assumed to be feed-forward.

2. Environments

In this chapter we will discuss the environments we have chosen to use in our examination of evolutionary algorithms and the theoretical background that will help us analyse them.

2.1 Reinforcement learning

Reinforcement learning might be best introduced by description of the problem it aims to solve.

Definition 7 (Markov's decision problem)**.** *Let P be a probability measure over the states of the environment. We define Markov's decision problem (MDP) by a four-tuple (S, A, T, r) where:*

- *S is a set of all possible states of the system, or state space*
- *A is a set of all possible actions available in the system or action space*
- *T is a transition function:*

$$\forall s', s \in S, a \in A : T(s, a, s') = P(s'|s, a)$$

- *$r : S \times A \times S \to \boldsymbol{R}$ is reward function, $r(s, a, s')$; being the reward received when the execution of action a in state s leads to state s'.*

To tie this into the information from chapter 1 $A = A(E)$, R is performance measure of an agent solving the MDP and the percepts have bijection to real states of the environment. This would not be the case for partially observable MDP.

Definition 8 (Kaelbling et al. [1998])**.** *Let P be a probability measure over the states of the environment. We define Partially Observable Markov's decision problem (POMDP) by a six-tuple (S, A, Ω, T, O, r) where:*

- *S is a set of all possible (true) states of the system, or state space*
- *A is a set of all possible actions available in the system or action space*
- *Ω is a set of all possible observations on the system, or observation space*
- *$T : S \times A \times S \to [0, 1]$: is a transition function over true states.*

$$\forall s', s \in S, \forall a \in A : T(s, a, s') = P(s'|s, a)$$

.

- *$O : S \times A \times \Omega \to [0, 1]$.*

$$\forall s' \in S, \forall a \in A, \forall o \in \Omega : O(s', a, o) = P(o|a, s')$$

or the probability of making observation o given that the agent took action a and landed in state s'.

- $r : S \times A \times S \to \boldsymbol{R}$ *is reward function same as in MDP.*

Now that we have characterised the environments we can apply to them concepts we have reviewed in the chapter 1. We begin by specifying the notion of percept sequence space from definition 1.

Definition 9 (Ross et al. [2014])**.** *Let* Ω *the space of all observations (*$\Omega \cong S$ *for MDP),* A *be the action space,* $o_t \in \Omega$ *be observation at time* t *and* $a_t \in A$ *action taken by the agent in time* t*. Then*

$$h_t = (a_0, o_1, ..., a_{t-1}, o_t)$$

is a history at time t*.*

Definition 10 (Ross et al. [2014])**.** *Let* P *be probability measure over states of the environment* S *and* $s_t \in S$ *be the state of the environment at time* t *. Then function* $b_t : S \to [0, 1]$ *such that*

$$b_t(s) = P(s_t = s|h_t, b_0)$$

is called a belief state of an agent, where b_0 *is the apriori belief state and* h_t *is history at time* t*.*

We also need to reframe the agent function discussed in definition 1.

Definition 11 (Ross et al. [2014])**.** *Let* P *be a probability measure and* A *be an action space. Then function* π *defined as* $\pi(b_t, a) = P(a|b_t)$ *where* $P(a|b_t)$ *denotes probability of doing action* a *given belief state* b_t *is called agent policy.*

Definition 12 (Ross et al. [2014])**.** *Let* r *be a reward function,* A *an action space,* T *a transition function,* $\gamma \in [0, 1)$ *a discount factor and* Π *be a set of all possible agent policies, then*

$$\pi^* = \arg\max_{\pi \in \Pi} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \sum_{s \in S} b_t(s) \sum_{a \in A} \sum_{s' \in S} r(s, a, s') T(s, a, s') \pi(b_t, a) \,\middle|\, b_0 \right]$$

is called optimal policy.

The goal of reinforcement learning is then to proximate optimal policy by trial and error, based on the received reward, as accurately as possible. This can be done utilising various online or offline methods such as Q-learning. In the literature and specifically in the cited Ross et al. [2014] the the definition 12 features γ as an discounting factor for the reward. This is a crucial parameter for reinforcement learning algorithms that require continuous rewards. However as we will discuss further in this chapter and chapter 3, in evolutionary algorithms we evaluate the policy as a whole and are only presented with final value calculated. Therefore it is not necessary to know or discover particular value of γ for the environments presented in this thesis.

Note that since in this thesis we are only concerned with simple reflexive agents, the belief state is not explicitly calculated statistic but is nonetheless reflected in the learned reactions to observation.

2.2 Gymnasium

We intend to set our agents into environments provided by Gymnasium project which emerged from its predecessor OpenAI Gym project. Both of them Python libraries, trying to provide researches with benchmark reinforcement learning problems in form of simple games, sometimes even recreating old console games such as Air Raid or Backgammon.

While Gymnasium provides wide range of environments, quite a number of them if not most do not provide their outputs in structured form. Instead the resulting screen has to be processed by user defined visual processor to extract information crucial for completing the task.

This introduces a layer of complexity that is outside of the scope of this thesis. Therefore we have decided to narrow our selection only to environments which do provide structured output. We describe the loop of receiving and providing feedback between agent and environment in algorithm 1.

Algorithm 1: Environment Interaction Loop

Input: Environment $\mathcal{E}$
Input: Number of episodes n
Input: Observation aggregation functions agg and $transform$
R = 0;
for i =*0* **to** n **do**
    $o \leftarrow \text{reset}(\mathcal{E})$;
    $O \leftarrow \text{reset}(O)$;
    while *not terminated* **do**
        $a \leftarrow \text{Agent}(o)$;
        $(o, r, terminated) \leftarrow \text{Environment}(a)$;
        $R \leftarrow R + r$;
        $O \leftarrow agg(O, o)$;
    end
end
return R/n, $transform(O)$;

As mentioned in the introduction the selected environments are called CartPole and Lunar Lander.

We will discuss the full purpose of agg and $transform$ in the section 3.2.2 and section 4.3.2.

2.2.1 CartPole

CartPole is one of the most used benchmark environments in the Gymnasium library. It is a simple environment where the agent controls a cart rolling on flat surface with a pole on top. The goal is to keep the pole balanced on top of the cart. The reward is calculated every step depending on the angle of the pole. When the pole is kept in the angle ± 0.418 rad for vertical position the agent receives +1 point. Since there are maximum 500 steps this means we have bounded total reward between 0 and 500 points.

Definition 13. *An agent that achieves score 500 in CartPole is called solution or solving individual.*

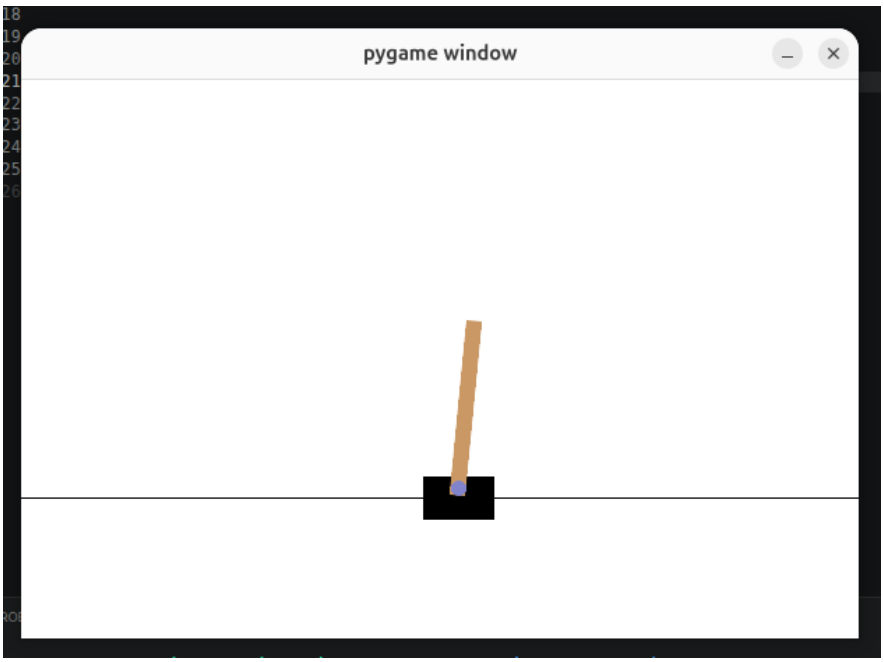

Figure 2.1: CartPole environment

Observation space

CartPole observation space is expressed as a 4-dimensional vector. We delineate possible values of the vector in 2.1. Note that for CartPole environment this observation space reflects full state of the environment in the given moment and therefore it could be characterised as MDP rather than POMDP.

Index	State Variable	Min	Max
0	Cart position	-4.8	4.8
1	Cart velocity	$-\infty$	∞
2	Pole angle	-0.418 rad	0.418 rad
3	Pole angular velocity	$-\infty$	∞

Table 2.1: Observation space of CartPole

Action space

The action space is discrete with only two values possible as presented in the table 2.2. Note that we cannot chose to do nothing.

Action	Description
0	Push cart to the left
1	Push cart to the right

Table 2.2: CartPole Action Space.

2.2.2 Lunar Lander

The other environment we have selected is Lunar Lander where the agent controls descend of a landing shuttle on rugged planetary (or the Moon's) surface with the help of oddly placed thrusters. The points are subtracted for staying in the air or for getting the shuttle into a spin. Bonus points are awarded for staying close in the centre of the arena. The shuttle has infinite fuel so the game ends only when the shuttle lands safely or crashes into the ground. We at least limit the number of steps for which the environment is evaluated. We set the limit to 120 steps. The limit has been chosen empirically during preliminary experiments. After setting the limit measured performance was not negatively affected.

For a neural network training this however poses a specific kind of problem. First, very narrow behavioural window that can achieve positive total reward. But mainly as opposed to the CartPole environment (2.2.1), now the unsuccessful runs may take significantly longer time than the successful ones. This means experiments are particularly costly for slowly converging algorithms. The upper bound on sum total reward is 200, the lower bound would possibly be $-\inf$, depending on the length limitation imposed. We therefore differentiate between several stages of obtaining a successful agent.

Definition 14. *An agent that achieves score 200 in Lunar Lander is called solution or solving individual.*

Definition 15. *An agent that achieves positive score in Lunar Lander is called well-behaved.*

Definition 16. *An agent that achieves an average score over 100 in Lunar Lander is called very well-behaved.*

Definition 15 stems from the fact that it can only be achieved if the agent does not crash the shuttle as doing so yields a penalisation of -100 points while not obtaining a boost of +100 points for safe landing, driving even a possible solution with score 200 to 0. Similarly for definition 16, it is highly unlikely to average score over 100 without at least once landing with perfect score 200.

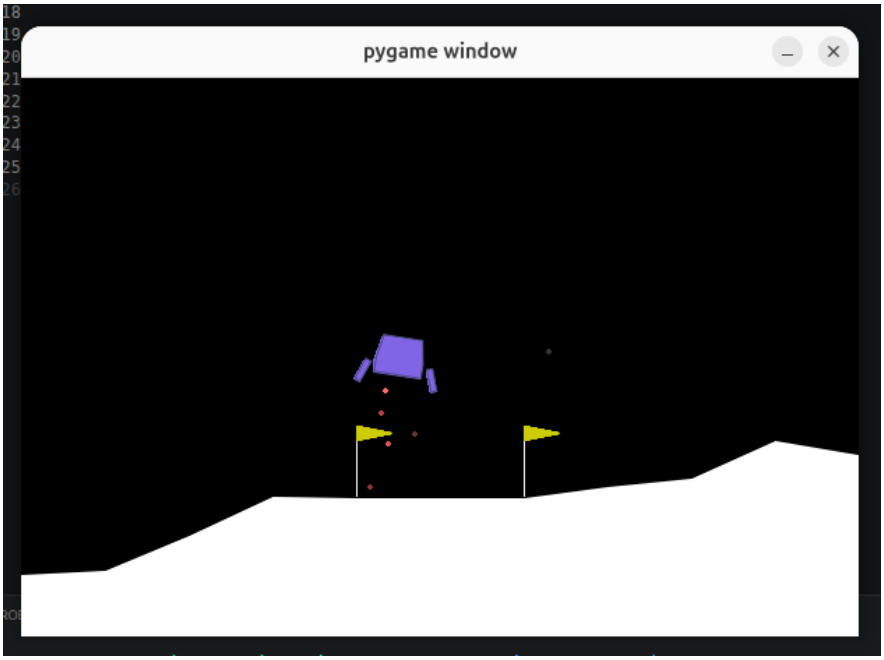

Figure 2.2: Lunar Lander environment

Observation space

The observation space of Lunar Lander is represented as eight-dimensional vector. Delineation of its values can be found in the table 2.3. Note that while the observation space is quite descriptive it does not capture all the information about environment's state directly such as wind velocity or proximity to the ground. Because the landscape (apart the landing pad in the centre) might be different in each episode, we can only infer their parameters from the observations (such as velocity and leg contact with ground) indirectly. For simplicity we do not allow wind in our episodes as is the default setting of the environment. This means that the distinction between observation and true state becomes most tangible in low altitudes, where agent risks either collision with unexpected hill or infinite hovering above unperceived valley, forgoing the +100 point reward.

Index	State Variable	Min	Max
0	x position	-2.5	2.5
1	y position	-2.5	2.5
2	v_x velocity in x-axis	-10	10
3	v_y velocity in y-axis	-10	10
4	angle θ	-6.2831855	6.2831855
5	angular velocity	-10	10
6	left leg contact (boolean)	0	1
7	right leg contact (boolean)	0	1

Table 2.3: Observation space of Lunar Lander

Action space

Action space for Lunar Lander has two possible variants. First is discrete in which the agent chooses a number representing which engine will fire. Second is continuous where agent has continuous control over the ignition of the main engine and one of the side engines. This variant allows for finer control during the flight but at the same time makes the policy the agent needs to learn significantly more complex, requiring more resources during the learning phase. To preserve compatibility with CartPole setup and considering feasibility of multiple experiments ahead we will work with the discrete version of the action space outlined in table 2.4.

.

Action	**Description**
0	Do nothing
1	Fire left orientation engine
2	Fire main engine
3	Fire right orientation engine

Table 2.4: Action space definition for the control system.

3. Evolutionary algorithms

The evolutionary algorithms (EA) are a class of optimisation methods modelled after our ideas of biological process of evolution. This chapter is based primarily on Introduction to Evolutionary Computing from Eiben and Smith [2003].

Definition 17. *General maximising optimisation problem takes the form:*

$$\begin{aligned} &\textit{maximise:} && f(x) \\ &\textit{subject to:} && g_i(x) \leq 0, \quad i = 1, \ldots, m, \\ &&& h_j(x) = 0, \quad j = 1, \ldots, p, \end{aligned}$$

where x is the solution vector, f is objective function,
and g_i, h_j are constraints imposed on x

Although theoretically it could be also minimising, in this thesis we will only encounter maximising optimisation and so our concern is mainly with the such case.

Because of the biological analogy, solution vectors are in the literature often referred to as *individuals* and we adopt this terminology throughout the thesis. Alternatively when emphasising either the numerical form of the vector or semantic interpretation of this form beyond the optimisation algorithm the individual may be referred to as either a **genotype** or **phenotype** respectively.
Group of individuals representing candidate solutions at a given time is referred to as a population. In general case evolutionary algorithms then evolve the population by applying three operations (operators) to it in a loop until either the population converges to the optimum or the limit on number of steps is reached.

Definition 18. *Let f be objective function in minimisation (resp. maximisation) problem. We call an individual i more successful than individual j if value $f(i) < f(j)$, resp. $f(i) > f(j)$.*

Algorithm 2: Generalised Evolutionary Algorithm

```
Data: Initial population = pop
Data: N = Number of generations
Result: Optimised population = pop
for g ∈ 1..N do
    co ← CrossOver(pop);
    m ← Mutate(co);
    pop ← Select(m);
end
return pop
```

Let us review the meaning of the aforementioned operators.

- crossover usually involves recombination of features from two or more individuals from the population. Its purpose is to provide means of exploitation of features beneficial to the population already developed by some individual.

- mutation provides means of exploration by introducing randomness into the individuals EA without crossover would be more akin to a random walk algorithm.

- selection then is about filtering the resulting individuals based on the objective function of the algorithm. It might be performed deterministically or with varying degree of stochasticity as elaborated in section 3.1.

It should be noted that the order of mutation and cross-over here are arbitrary and are specific to each particular evolutionary technique. For further discussion of selection process we need to introduce a few terms specific to evolutionary algorithm that have been borrowed from their counterpart in biology.

3.1 Continuous Optimisation

Historically the individuals optimised by EA used be primarily conceived as vectors over binary domain - $\{0, 1\}$. However this approach was later expanded into the domain of vectors over real numbers. In literature this is referred to as continuous optimisation. In this thesis we are interested in neuroevolution, a subset of continuous optimisation as neural networks discussed in the chapter 1 are represented as a series of vectors of real numbers. But since the aim of this thesis is mainly to compare different objective functions we will not delve too deeply into the intricacies of neuroevolution field and instead approach the following techniques from the more general standpoint of continuous optimisation.

In this section we will review the selected evolutionary algorithms and describe them as they are going to be used during the experiments in chapter 5.

3.1.1 Evolutionary Strategies

We base information in the section mainly on the [Beyer and Schwefel, 2002] paper. Evolutionary strategies (ES) are very direct incarnation of algorithm 2 into the realm of real numbers.

We recognise two types of evolutionary strategies: $\mu + \lambda$ and μ, λ. μ denotes size of already existing population while λ denotes size of new population pool. In $\mu+\lambda$ selection happens from the existing population pool and the new population pool at the same time while the μ, λ variant only ever selects from the new population pool. This makes the evolution $\lambda + \mu$ more stable as established successful individuals have a bigger chance to pass to the next generation unless a more successful individual has been found. That said, for some problems this approach risks stagnation and it is preferable to use μ, λ. However the experience with our selected environments from chapter 2 clearly showed dominance of $\mu+\lambda$ as even for the CartPole, simpler of the environments, improving mutations were too costly to be lost on an unlucky new generation.

Therefore all further discussion of evolutionary strategies will be focused on $\mu + \lambda$. We detail the procedural aspects of $\mu + \lambda$ in the algorithm 3.

Mutation

Distinct feature of evolutionary strategies is the mutation operator that takes as a parameter standard deviation σ and which mutates the value stochastically by a number $y \sim N(0, \sigma)$. In the literature we can often find ES where the mutation σ is part of the individual genotype this might help the evolution as more successful individuals might enforce milder mutation while less successful individuals might mutate wildly in search of better genotype. For our purposes this however introduces two problems. Firstly it complicates interpretability of the results and since we are more interested in the comparison than raw results we cannot ignore this downside. Secondly this approach intuitively works well for cases where objective function is also the meta-measure we actually care about but we are precisely interested in comparison with cases where the objective function is rather orthogonal to actual usefulness of the individual as we will discuss in section 3.2. There this approach could be chaotic at best and detrimental at worst.

For these reasons we opt for a global σ that does not change during the run of the evolution.

Crossover Methods

Crossover is a method of combining values from two selected individuals. Based on We will discuss mainly two kinds of cross-over:

- **Arithmetic** crossover where we combine genotype vectors x_1, x_2 into a new genotype x' as follows:
$$x' = \frac{x_1 + x_2}{2}$$
- **Uniform** crossover where iterate over components of two parent genotype vectors x, y through index $i \in 1 \dots d$ where d is the size of the genotype (we assume both parents are the same length). Then with parameter p_u we produce new individual's genotype v like so:
$$v_i = \begin{cases} x_i & \text{if } r \sim \text{Uniform}(0,1) < p_u \\ y_i & \text{otherwise} \end{cases}$$
.

We do not consider position based cross-over methods as they might become quite difficult to manage for the neuroevolution purposes.

Tournament selection

Selection in evolutionary algorithms can be realised by various means such as roulette selection where each individual i has likelihood of passing into the next generation p_i as follows: $p_i = \frac{f(i)}{f(pop)}$, where $f(i)$ is objective function value for the individual i and $f(pop)$ is sum of objective value of all individuals. This however works only for objective functions that give out only either negative or positive numbers. As we have described in chapter 2 environments explored in this thesis can give out rewards with value negative or positive based on the agent's performance. Therefore we will utilise tournament selection described in Goldberg and

Deb [1991]. Tournament selection works on simple principle where we select pool of individuals, size of which is called *tournament size*. From them we then select the one with the most optimal objective function value [Goldberg and Deb, 1991]. We detail the workings of tournament selection in algorithm 3. Note that for the purposes of this thesis we have selected *tournament size* = 3, or ternary tournament. During preliminary experiments using binary tournament we have often lost valuable individual to the instability of the selection. Ternary tournament provided more stability of the evolution while maintaining the indeterministic nature of the algorithm.

Algorithm 3: Evolutionary Strategy $\mu + \lambda$

Input: Population size μ, new generation pool size λ, crossover rate CR, mutation rate MR, objective function $f(x)$, crossover operator $CrossOp(x)$, mutation power σ, number of generations G

Initialise population $\{x_i^0\}_{i=1}^{\mu}$;
Evaluate $f(x_i^0)$ for all i;
// x^0 forms initial population
for $g = 0$ **to** $G - 1$ **do**
  for $i = 1$ **to** λ **do**
    // Crossover:
    randomly choose $r_{CR} \sim Uniform(0, 1)$;
    if $r_{CR} < CR$ **then**
      Select random individuals $a, b \sim Uniform(1, \mu)$;

$$v_i \leftarrow CrossOp(x_a^g, x_b^g)$$

    else
      Select random individual $a \sim Uniform(1, \mu)$;

$$v_i \leftarrow x_a^g$$

    // Mutation:
    randomly choose $r_{MR} \sim Uniform(0, 1)$;
    if $r_{MR} < MR$ **then**

$$v_i' = v_i + \sigma\mathcal{N}(0, 1)$$

    else

$$v_i' = v_i$$

    Evaluate $f(v_i')$;
  // v' forms the new population pool
  $v'i \rightarrow x_{\mu+i}^g$;
  // Tournament selection:
  for $i = 1$ **to** μ **do**
    Select random individual $a, b, c \sim Uniform(1, \mu + \lambda)$;

$$x_i^{g+1} \leftarrow \arg\max(f(x_a), f(x_b), f(x_c))$$

return x^G

3.1.2 Differential Evolution

In the introductory paper Storn and Price [1997] on which this section is mostly based, the differential evolution is characterised mainly by its industrial capabilities such as easy parallelisation and application.

Because of these benefits it is a popular technique in the field of neuroevolution. We detail the whole process in algorithm 4. The algorithm for resolving a single generation happens in a single loop with the familiar stages which we detail in algorithm 4.

Selection

Unlike in evolutionary strategies described in section 3.1.1, selection is done on individual basis to enhance parallelisation capabilities of the algorithm. This means that for each individual x_i we either select it or its proposed replacement v_i based on the values of objective function for each of them. Of course we keep the more successful one.

Mutation

While Evolutionary strategies introduce stochasticity in explicit and controlled manner, differential evolution relies on a special form of contextual mutation where three individuals from the population are drafted and combined. This means that we do not have to search for optimal value of σ.

We select three individuals x_a, x_b, x_c distinct from x_i and compute a new individual: $x'_i = x_a + \alpha(x_b - x_c)$ where x_a is called anchor, $(x_b - x_c)$ differential and α is scaling factor.

Although the scaling factor α does not have probabilistic interpretation, in some sense it corresponds to mutation rate in ES as the greater it is, the more stochasticity affects the system.

Recombination

During recombination we decide for each vector component of the newly forming individual x'_i whether we will source it from the already mutated version x'_i or the original x_i based on recombination rate γ. This implementation differs from the standard formulation of Differential Evolution in that the recombination does not enforce transfer of at least one component from the mutated vector. Consequently, there is a non-zero probability that the recombined vector is identical to the target vector. This may have negative effect on the ability of the search to generate diverse individuals and consequently make it less effective. Since the choice of algorithms is mostly arbitrary this does not affect validity of this thesis but it should be taken into consideration when interpreting the results. All experimental results presented in this thesis were obtained using this implementation. Recombination is rough analogue of crossover for the purposes of single easily parallelised loop and recombination rate could be likened to the uniform crossover cross-rate R_U.

Algorithm 4: Differential Evolution

Input: Population size N, scaling factor α, recombination probability γ, objective function $f(x)$, number of generations G

Initialise population $\{x_i^0\}_{i=1}^N$;
Evaluate $f(x_i^0)$ for all i;
for $g = 0$ **to** $G - 1$ **do**
  for $i = 1$ **to** N **do**
    `// Mutation:`
    Select random individuals $a, b, c \neq i$;

$$v_i = x_a^g + \alpha(x_b^g - x_c^g)$$

    `// Crossover:`

$$u_{i,j} = \begin{cases} v_{i,j} & \text{if } r \sim \text{Uniform}(0,1) < \gamma \\ x_{i,j}^g & \text{otherwise} \end{cases}$$

    `// Selection:`
    Evaluate $f(u_i)$;
    if $f(u_i) \geq f(x_i)$ **then**
      $x_i^{g+1} \leftarrow u_i$;
    else
      $x_i^{g+1} \leftarrow x_i^g$;
return x_i^G

3.1.3 Archives

A technique that may enhance the techniques presented in this section is incorporating an archive into the algorithm. Archive is a long-term storage outside of the optimised population of a certain size where a batch of individuals selected from regular population is added periodically and are reintroduced during the selection. This approach can help preserve diversity of individuals or on the contrary to preserve the best solutions found so far. We will distinguish archives intended for the second purpose as *elite archives* .

For evolutionary strategy we simply add the archive batch from which a new generation pool is being selected as this will not affect the run of the algorithm in any way.

For differential evolution we cannot just add the archive to the existing population. We actually have to reintroduce the the archived individuals instead of parts of the population. In this thesis we will do this on basis of random pick from the *population* + *archive*.

3.2 Objective functions

In this section we will discuss possible objective functions for the evolutionary optimisation search.

3.2.1 Fitness

Using objective function as the utility function of the algorithm is the most direct way of establishing relationship between the solution we want and the individuals in the evolving population.

Though this approach ensures that the most optimal individuals from given population will generally be selected into the next generation during the evolution it does not guarantee that their potential for evolving is not diminished either. This way the evolution using fitness can in easily become stuck in local optimum of the objective function. While in most cases it might be still possible to reach global optimum on the given problem by changing the hyper-parameters to favour random exploration, it may come to point where the benefits of evolutionary approach are made void.

Archiving

As stated in section 3.1.3 for fitness the two main reasons for introducing archive into an fitness based EA is either to promote diversity by randomly preserving part of the population which might have been eradicated to the detriment of the overall evolution or on the contrary to preserve the best performing individuals so they may not be accidentally selected out of the gene pool.

3.2.2 Novelty

In its introductory paper novelty Lehman and Stanley [2011] is conceived as a guide of the evolutionary search to help avoid traps of local optima.

Definition 19 (adapted from [Doncieux et al., 2019])**.** *Let E be environment and h_t be a history up to a time t and let $d(x, y)$ be a metric over set B . Then (B, d) is a behavioural space if there is a function $\omega_B : h_t \to B$, called observer function.*

For the purposes of this thesis the observer function is implemented through the $agg(x)$ and $transform(x)$ functions used in algorithm 1.

Definition 20 (adapted from [Lehman and Stanley, 2011])**.** *Let P be a population of individuals (their histories) within EA and (B, d) be a behavioural space. Then for each individual a we define novelty:*

$$N(a) = \frac{\sum_{b \in P, b \neq a} d\left[\omega_B(a), \omega_B(b)\right]}{|P| - 1}$$

With novelty as objective function the selection process incentivises diversification of behaviour among the population. This allows the evolutionary search to retain variety within the population across generations, leveraging the crossover as well as mutations to leave local optima.

Downside of this approach might be in the ambiguity in the choice of behavioural space. Incorporating too many environment variables will lead to faux-exploration in variables completely orthogonal to fitness. On the other hand leaving out an important variable means the algorithm will not retain diversity where it matters, making novelty pointless. This means the behavioural function and space have to be selected empirically for given problem.

Our implementation of novelty is different from the original paper in that we compute the average of distance to the whole reference population. This is done to simplify the computation as the size of populations in this thesis are not particularly large (< 100). If we do not state otherwise in this thesis novelty will be calculated in relation to the upcoming generation $g+1$ During the unrecorded preliminary experiments we have not found any significant advantage to other methods of computation.

Archiving

In fitness the random archive had a role of providing variety, which is the main goal of novelty search. Here we actually hope to use the randomness as a stabilising element since reintroduction of old behaviours will prompt the new generation to avoid them. This approach was with success used in Gomes et al. [2015], outperforming pure novelty and novelty based archive.

To implement elitist version of archive for novelty we will introduce a fitness threshold T_f to the random archive so that given a randomly selected batch Q_{rand} and fitness function f we screen the selected individuals so that we get actual elite batch Q_{elite}:

$$Q_{elite} = \{i \in Q_{rand} | f(i) \geq T_f\}$$

Only these screened individuals are then added to the archive. This will help us explore the idea of the archive's stabilisation role.

3.2.3 Combinations of Novelty

The novelty does not take into account the individual's performance with objective function. Therefore when two individuals with similar behavioural characteristics appear but one of them performs significantly better than the other, pure novelty algorithm can easily select the less performant individual, losing the more performant genome for further generations. To prevent this we want to meaningfully combine novelty with fitness.

Exponential Decay Weight

Taking inspiration from Cuccu and Gomez [2011], we will try to combine novelty with fitness in the most straight-forward way - linear combination with decaying weights. However first we need to address some issues such as the disproportion in scale of fitness to scale of novelty. This means we require normalisation for the two. Again taking inspiration from Cuccu and Gomez [2011], we will normalise with the maximum and minimum of both fitness and novelty in the currently

existing population for each individual i:

$$\hat{f}_i = \frac{f_i - f_{\min}}{f_{\max} - f_{\min}} \mid f_{\min} \neq f_{\max}, \qquad \hat{n}_i = \frac{n_i - n_{\min}}{n_{\max} - n_{\min}} \mid n_{\min} \neq n_{\max}$$

In other the case where minimum and maximum are equal we set the respective value to 0. This is done so that homogenous populations can be more easily disrupted by the algorithm.

Both maximum and minimum are easy to obtain as they are one O(n) while other methods such as z-score might be computationally more demanding in the multiplicative constant. Then we get combination:

$$c_i = W\hat{f}_i + W'\hat{n}_i$$

Unlike Cuccu and Gomez [2011] we will however decay the weight W of fitness or $W' = (1-W)$ of novelty over time. So that for base W_b, decay rate r_D, current generation g, total number of generations G and **increasing novelty** we have:

$$W = W_b e^{-r_D \frac{g}{G}}$$

Conversely for **increasing fitness**

$$W' = W_b e^{-r_D \frac{g}{G}}$$

This will allow us to explore whether the exploratory stage has more meaning at the beginning when it generates diverse population that can later pruned by the shift for fitness or is it better to first evolve the population to certain level and then explore with more adept individuals.

4. Comparison criteria and conditions

To compare different types of objective functions fairly, we have to select criteria by which we compare them and establish conditions under which will this comparison take place so that our conclusions are fair.

4.1 General criteria

Since fitness provides direct measure of performance we would like to make our comparison on the fitness of the final population and its distribution in it. To that end we have to perform experiments with variety of initialisations of the respective environments, getting more comprehensive picture than a single run. This will be mainly reflected in the seed of random numbers generator of the environment. Since the main advantage of the novelty approach lies mainly in avoiding the trap of local optima we also want to review statistics of each generation produced during the algorithm runs
However we also have to take into account the hyper-parameters with which these populations have been achieved. This is especially (but not limited to) population size and number of generations, hyper-parameters responsible for most of the computational complexity of an evolutionary algorithm run.

4.2 Containers

While we have reviewed evolutionary algorithms and proposed different objective functions in chapter 3, we now have to somehow refer to the combination of the two. We use the term **container** to denote a concrete combination of an evolutionary algorithm, objective-handling method. We can divide the containers on the axis of algorithm to ES containers and DE container. And we can divide them on basis of objective function handling:

- **fitness** container implements pure fitness approach as described in section 3.2.1
- **novelty** container implements pure novelty approach as described in section 3.2.2
- **fitness archive** container implements fitness search with random selection archive as described in section 3.2.1
- **elite archive** container implements fitness search with elite selection archive as described in section 3.2.1
- **novelty archive** container implements novelty search with random selection archive as described in section 3.2.2

- **limit archive** container implements novelty search with fitness threshold archive as described in section 3.2.2
- **increase novelty** container implements increasing novelty objective as described in as described in section 3.2.3
- **increase fitness** container implements increasing fitness objective as described in as described in section 3.2.3

Specifically for ES the crossover method is considered a hyperparameter that will be pinpointed during the Bayes search described in section 4.4.1.

4.3 Individual hyper-parameter selection

The hyper-parameters characterising individuals are in essence parameters of the neural networks constituting them: number of layers and number of neurons in the layers. The choice of these parameters is specific for each task. Generally the more complex the task the more of either layers or neurons per layer the handling NN requires to perform well.

4.3.1 NN Structure

As discussed in chapter 1 we have committed to making our agents be simple feed-forward neural networks. This choice is on account of simplicity but also on account of adequacy. The bigger the neural network is the more resources need to be spent on optimising it. The NN structures are depicted on figures 4.1 and 4.2 for CartPole and Lunar Lander respectively.

Both structures have been selected on account of preliminary experience with the environments. Shapes of input and output layer are determined by the respective observation and action space.

The genome of evolving individual consists purely of the weight and bias matrices of the hidden layers. Nothing else is subject to evolution. For the purposes of this thesis and in relation to the descriptions of evolutionary mechanics given so far we should think about these matrices as flattened to a single long vector. In the case of Lunar lander with multiple layer the vectors are concatenated.

4.3.2 Behavioural Space

As we can see through the definition of observer function in definition 19, the choice of transformation from history to behavioural space can be arbitrarily complex. Recently there have been proposals of automatic behavioural space extraction like in the paper Meyerson et al. [2016] or Salehi et al. [2021] . However the original paper Lehman and Stanley [2011] emphasises application of domain knowledge to the behavioural space design. Because this thesis is in the core comparative and we are interested in the interpretability of the design as much as raw performance, we have opted for manual design based on our preliminary experiments and experience with the environments. We also take into account the fact that the greatest advantage from an automatic behavioural space extraction is obtained mainly in more complex environments [Meyerson et al., 2016].

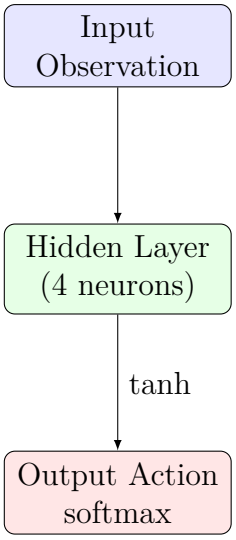

Figure 4.1: Structure of CartPole agentic network

While we won't specify agg and $transform$ functions introduced in algorithm 1 explicitly for readability purposes, their definitions are implied by respective definition of observer function ω_B and the fact we will only consider the behaviour of the last evaluated episode. While such approach loses some of the information about the evaluated individual it also does not introduce consistency trade-offs of standard aggregations like max, min or *mean* applied component-wise. Addressing these trade-offs is beyond the scope of this thesis.

CartPole

Let $h_{Cart} = (a_0, o_1, a_1, \ldots, o_T)$ be history of one completed episode of CartPole that lasted T steps and o_T is the last observation. Then we define observer function:

$$\omega_B(h_{Cart}) = (pos_X(o_T), \theta(o_T))$$

where $pos_X(o_T)$ is reported position on x-axis in observation o_T and $\theta(o_T)$ is pole angle, both normalised to interval $[-1, 1]$ through their respective maximal values reported in table 2.1.

The reasoning behind this choice of behavioural space is that the combination of position and pole angle is indicative not only of the past performance but also of the ability to continue the task in the next round. Therefore searching through the space of these behaviours should eventually yield performant individuals.

Lunar Lander

Let $h_{Lun} = (a_0, o_1, a_1, \ldots, o_T)$ be history of one completed episode of Lunar Lander that lasted T steps. Then we define observer function:

$$\omega_B(h_{Lun}) = (pos_X(o_T), v_Y(\bar{o}))$$

where $pos_X(o_T)$ is reported position on x-axis in observation o_T, v_Y is a reported velocity in y-axis and $\bar{o}$ is mean of all observations recorded in h_{Lun}. Both components of behavioural vector again normalised to interval $[-1, 1]$ through their respective maximal values reported in table 2.3.

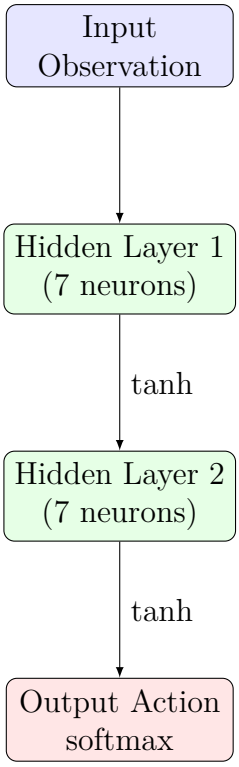

Figure 4.2: Structure of Lunar Lander agentic network

The reasoning behind this choice of behavioural space is that the combination of average speed of descend and landing position is indicative of the agent's ability to descend safely and control over the descend. Searching over this space should therefore eventually produce performant individual.

4.4 Algorithm hyper-parameter selection

We have reviewed hyper-parameters of each algorithm in the chapter 3. Now we will review the process of their selection. The difficulty of the task lies mainly in the complexity of evaluation. As we will discuss later some of the experiments maybe evaluated over day or even several days on the available hardware even for a 100 limited algorithm runs. For this reason we need to utilise Bayesian search which is able to eliminate much of the redundant space out so that we may not waste time with it. This however has a considerable downside for our purposes as we would like to compare the algorithms on their best footing. Therefore we refine the Bayesian search results with local grid search attempting to deviate from the supposed hyper-parameter equilibria.

4.4.1 Bayesian Search

Because we are faced with multiple Bayesian search is an algorithm where we stochastically delineate where a possibly good solution may lie and then try to exploit this delineation. Because the specific workings of Bayesian search are time-consuming to implement ad hoc (and also such implementation is out of the scope of this thesis) We will utilise library Optuna.

Optuna

Just as in our case a lot of machine-learning techniques need for their proper function to obtain adequate hyper-parameters first. Optuna is a Python library designed for this purpose [Akiba et al., 2019]. The library itself is multifaceted, so we will only discuss algorithms and terms pertaining to the subject of this thesis.

Definition 21. *By Optuna trial or just trial we mean one run of the evolutionary algorithm with one chosen set of hyper-parameters and the results of this run.*

Definition 22. *Optuna study is a set of related trials with implied continuity of choice.*

Definition 23. *Optuna optimisation run describes run of the chosen optimisation algorithm, adding optimised trials to selected study.*

Note that study maybe composed of multiple optimisation runs. In this thesis we have only done one study per container and one optimisation run per study if not for no other reason than to mind our computational time.

As the optimisation algorithm we have selected Tree-structured Parzen Estimator (TPE) due to the fairly low complexity and effectiveness, wasting no trials as we want to conserve as much actual runtime as possible.

Tree-structured Parzen Estimator

In this section we will use information from paper Bergstra et al. [2011]. The Tree-structured Parzen Estimator (TPE) used in Optuna is a sequential model-based optimisation method for hyper-parameter tuning. It models probability $p(x|y)$ where x is a set of hyper-parameters and $y = f(x)$ is observed value of objective function.

It does so by learning two marginal models $l(x)$ a $g(x)$ This split $y*$ defines what the algorithm currently considers promising based on past experience. The threshold $y*$ is set to some quantile of observed objective values.

$$p(x|y) = \begin{cases} l(x) & y < y* \\ g(x) & y \geq y* \end{cases}$$

Model $l(x)$ represents density learned from observations $\{x_i | f(x_i) < y*\}$ and model $g(x)$ represents density learned from observations $\{x_i | f(x_i) \geq y*\}$.

These distributions are typically estimated using smooth non-parametric density estimators (Adaptive Parzen estimators), which can flexibly model hyper-parameters without strong assumptions about the shape of the space. The next tried set of hyper-parameters is chosen according to the rule:

$$x^* = \arg\max_x \frac{l(x)}{g(x)}.$$

4.4.2 General Gridsearches

Now we will discuss grid searches that are common for all implementations of our EAs. Note that for convenience we do not present these concepts in the temporal order in which they have been evaluated.

Evaluation episodes vs. generations

Before we start other experiments we have to determine how many episodes will be used to evaluate an individual during the algorithm's run in these experiments. The parameter directly influences how much information the algorithm has extracted about the individual, as more episodes will lead to more indicative behavioural descriptors. However since the episodes take up non-trivial part of the computation we want to select the lowest possible number. For this purpose we will run a grid search comparing differential algorithm's performance over different number of generations. Since the Gymnasium environments discussed in chapter 2. are based on similar principles we will perform the search only and CartPole but generalise the results to both. This is done to conserve time for more impactful experiments.

Obtaining the values from the grid search we will select the optimal number of episode based on the following rationale: For each episode count tested we define strategies S where $s \in S$ is the corresponding number of episodes. We name expected performance of strategy s in run of length g as $\mathbb{E}[V_g(s)]$ We define gain of strategy s in run of length g as $A_g(s) = \mathbb{E}[V_g(s)] - \mathbb{E}[V_g]$ where $\mathbb{E}[V_g]$ is the total expected performance in generation g across all strategies. We then select

$$s* = \arg\max_{s \in S} \left(\frac{\mathbb{E}_{g \sim G}[V_g(s)]}{cost(s)} \right)$$

where $cost(s) = e^s$, introducing exponential cost to strongly penalise increasing episode count, accounting for the overall difficulty associated with the increase.

Generations grid search

During the refining in Bayesian search and Incremental grid search we use fixed number of generations for the optimisations. This means we don't know how many generations are truly best. We perform a grid search over a range of generations. This will also allow us to compare performances and performance curves based on this parameter.

4.4.3 Incremental grid search

Because we obtained the optimised parameters from Bayesian search stochastically we would like to ensure that there is no easily obtainable set of parameters more performant than the set from BS. All however in time consuming fashion. To this end we have devised an algorithm of incremental grid search where we evaluate possibly useful deviations from the supposed local optima. Upon discovering new optima we recentre the search around it. If we are not successful in finding dominating set of parameters we end the search with the last obtained optima. We also end the search after more than 3 re-centring (hops). Since this

is only refining algorithm of already functional solution we do not expect long searches.

Algorithm 5: Incremental grid search general

Input: Initial argument set x_0, number of hops H,
evaluation algorithm A
Output: Final argument set x

$x \leftarrow x_0$;
$\mathcal{V} \leftarrow \emptyset$;
`// fitness performances`
$s \leftarrow \emptyset$;
`// final populations`
$pop \leftarrow \emptyset$;
$s(x_0), diversity(x_0) \leftarrow A(x_0)$;
for $h \leftarrow 1$ **to** H **do**
  Generate valid candidate argument sets $\mathcal{C} \leftarrow \text{Generate}(x, \mathcal{V}, h)$;
  foreach $c \in \mathcal{C}$ **do**
    $s(c), diversity(c) \leftarrow A(c)$;
    `// for` $c \in \mathcal{V}$ `we use cached values`
    $\mathcal{V} \leftarrow \mathcal{V} \cup \{c\}$;
  if $\mathcal{C} \neq \emptyset$ **then**
    $S^\star \leftarrow \max_{c \in \mathcal{C}} s(c)$;
    $\mathcal{B} \leftarrow \{c \in \mathcal{C} \mid s(c) = S^\star \wedge s(x) \leq s(c)\}$;
    if $|\mathcal{B}| = 1$ **then**
      $x \leftarrow$ only element of $\mathcal{B}$;
    else if $|\mathcal{B}| = 0$ **then**
      `// We have not found improving solution`
      return x, $s(x)$;
    else
      $x \leftarrow \arg\max_{c \in \mathcal{B}}(diversity(c))$;
return x, $s(x)$;

The candidates are generated from neighbourhoods of the selected solution x in one or more hyper-parameter. For hyper-parameter H the neighbourhood is defined by δ_H. We generate $x_{-\delta}$, $x_{+\delta}$, x_0 such that for $x_{-\delta}(H) = x(H) - \delta_H$ and analogically for $x_{+\delta}$. The option of x_0 might not make sense to generate for a single hyper-parameter as this is just x repeated but in combination of two or more hyper-parameters this ensures we truly check all combinations consisting the neighbourhood of x in those hyper-parameters.

4.4.4 $\mu + \lambda$ incremental grid search

For the purposes of $\mu + \lambda$ algorithm we split the Generation and evaluation of candidate solution for the iteration of the main loop into two parts. First part generates candidates in the neighbourhood on the hyper-parameters $\mu + CR$, the second $\lambda + MR$. While all hyper-parameters interact we claim that theses pairs interact very strongly if not strongest as exploitation power of CR is directly interlinked with the stochastic diversity reservoir of the current population. On the other hand the size the new individual pool limits exploration power of mutation rate. It should be noted that the selection process happens after both candidate

batch generations and so the candidates in $\lambda+MR$ stage might be generated from solution already updated by $\mu + CR$ stage. The reason why we do this instead of generating all at once is time conservation and priority based on projected stability of the solution - we claim that alterations to the main population and crossover rate produce more stable algorithm based on preliminary experimentation. If we don't find the solution at first stage algorithm cannot prematurely terminate before the second stage is evaluated negatively as well.

4.4.5 DEs incremental grid search

As we described in section 4.4.4 we split the candidate generation even for the differential evolution. First stage alternates only size of the population. Second stage alternates hyper-parameters $\alpha + \gamma$ or the rescale factor and recombination rate introduced in 4. The algorithm again cannot terminate prematurely before both stages have been checked for improving solution.

4.5 Final comparison

In addition to graphical methods suggested in paper Beiranvand et al. [2017] to examine the results of the test we will use several statistical methods. Following advice in the paper García et al. [2009], which is specifically concerned with evolutionary algorithms, we will mainly utilise the non-parametrical tests which we introduce in this section. This enables us to compare results of our algorithm without imposing additional constraints.

The tests represent a null hypotheses H_0 which may or may not be rejected in favour of hypotheses H_1. This is done based on obtained p-value p representing a probability of observing the obtained data assuming the null hypothesis is true. We define a threshold of significance α so that for $\alpha > p$ the null hypothesis is rejected. Usually values $\alpha = 0.1$ or $\alpha = 0.05$ are chosen.[García et al., 2009]

4.5.1 Friedman test

We will use description of Friedman test from García et al. [2009]. There it is described as a statistical test testing k algorithms. The null hypothesis is that all k algorithm sample from the same population distribution. If rejected we accept at least one of the k algorithms samples from a different population distribution from the rest. We compute rankings r_j^i of each algorithm j for each comparison instance i. In the case of this thesis comparison instance is a initial conditions set for an particular instance of evolutionary algorithm. Rank is distributed from the best having rank 1 to the worst rank k. Then we compute the Friedman statistic χ_F^2:

$$\chi_F^2 = \frac{12N}{k(k+1)} \left(\sum_{j=1}^{k} R_j^2 - \frac{k(k+1)^2}{4} \right)$$

where $R_j = \frac{1}{N} \sum_{i=1}^{N} r_j^i$ and N i the number of comparison instances. The χ_F^2 is then converted to p-value through a combinatorial computation. In reviewed literature it is not common to go over the computational process explicitly as it

maybe quite complex in the variety of approximations. We will use the implementation in Python library SciPy which cites Spencer et al. [2025].

4.5.2 Wilcoxon signed-rank test

Introduced in the paper Wilcoxon [1992] in both paired and unpaired version, Wilcoxon signed-rank test enables us to compare two algorithm without assumptions about the distributions of their tested output. We start by computing differences of the algorithms' A and B performances in each instance i of of total of N like so:

$$d_i = A_i - B_i \tag{4.1}$$

Unlike in Friendman test where the rank is distribute across algorithms, in Wilcoxon test distributes rank across the absolute value of the differences $|d_i|$. Then we compute sums of ranks testifying for each of the compared algorithm. R^+ is the sum of ranks where algorithm A won ($d_i > 0$) while R^- is the sum of ranks where algorithm B won ($d_i < 0$). For $d_i = 0$ we split the ranks between the two sums [García et al., 2009].

$$R^+ = \sum_{d_i>0} rank(d_i) + \frac{1}{2}\sum_{d_i=0} rank(d_i)$$

$$R^- = \sum_{d_i<0} rank(d_i) + \frac{1}{2}\sum_{d_i=0} rank(d_i)$$

According to the paper García et al. [2009] we then obtain value

$$T = \min(R^+, R^-)$$

.

The T is then compared to the value of Wilcoxon distribution with N degrees of freedom. The null hypothesis that both algorithms perform equally is rejected if the T is less or equal. Corresponding p-value is calculated using normal approximation of Wilcoxon T statistic. [Cureton, 1967]

Holm's and Hochberg adjustment

Since we will be performing number of comparisons between different algorithm instances, we need to take measures to prevent family-wise error rate (FWER). To do so García et al. [2009] recommends Holm's and Hochbergs adjustment procedures.

Holm's procedure orders the obtained p-values so that: $p_1 \leq p_2 \leq \cdots \leq p_{k-1}$. Each p-value p_i, starting with the most significant, is then compared to $\frac{\alpha}{k-i}$ where α is significance threshold. We continue up the chain as long as we can reject the null hypothesis H_i based on $\frac{\alpha}{k-i}$. We stop once we cannot reject the null hypothesis and we conclude that all subsequent null-hypotheses hold up. [García et al., 2009]

In Hochberg's procedure we start from the least significant p-value and work our way down the chain. For the chain $p_1 \geq p_2 \geq \cdots \geq p_{k-1}$ comparing p_i with $\frac{\alpha}{i}$. If the null hypothesis holds up we move on to the next p-value. If it is rejected we consider all subsequent null-hypotheses rejected as well.[García et al., 2009]

Hodges–Lehmann estimator

Hodges-Lehman estimator is method of estimating direction of relationship between two treatments tested by Wilcoxon test published in Jr. and Lehmann [1963]. Taking the differences from eq. (4.1) we calculate the Hodges–Lehmann estimator HL for N differences d_i as such:

$$HL = \text{median}\left(\left\{\frac{d_i + d_j}{2}\middle| 1 \leq i \leq j \leq N\right\}\right)$$

This method should then provide more accurate picture of the comparison than simple median of differences [Jr. and Lehmann, 1963].

5. Experiments

In this chapter, we will discuss the experiments and their results. The code for the experiments has been written in Python 3.10 and experiments were conducted on several computers characterised in section A.4. The complete project with data and scripts is available publicly in Github repository.

5.1 Episode Grid Search

A goal of this experiment is to determine the number of episodes for which we should evaluate an individual on an environment for the purposes of evolutionary selection.

As we have described in the section 4.4.2, to do so we have performed a grid search on the CartPole environment using the pure fitness method on both evolutionary algorithms (ES and DE). We performed the experiment with differing lengths of algorithm run (in generations). The remaining hyper-parameters were selected based on preliminary experiments which we have set as respective default values in the codebase.

The generations and episodes over which the search was conducted are

$$g \in \{3, 7, 10, 15, 20, 30\}$$

$$s \in \{1, 2, 3, 4, 5, 6, 7\}$$

5.1.1 Results

The aggregated results based on the calculation introduced in section 4.4.2 are presented in table 5.1, number of episodes on the left and calculated benefit/cost ratio on the other.

eps	cost adj. gain
1	-8.320725
2	0.153129
3	**0.317113**
4	0.213275
5	0.078688
6	-0.021315
7	0.000359

Table 5.1: Evaluation episode selection table. Left is episode count, right is the calculated utility value as per maximising expression in 4.4.2

5.1.2 Discussion

Based on our calculation of utility we have selected the length of evaluation within the evolutionary algorithm to be 3 episodes. The calculation is expression of our preferences within this situation so while greater episode counts might technically yield better results they are not better enough to justify the slowdown of the

algorithm run. Most of the algorithm's run is constituted by evaluation of the environment. For evaluation of produced population at the end of EA's run we will use 5 episodes as the operation is done only once at the end of an run and the time cost is not as punishing in overall picture.

5.2 Bayesian Search

The goal of this experiment is to obtain a set of optimised hyper-parameters specific to the algorithm and objective handling method. As mentioned in section 4.4.1, we perform one optimisation run per container, resulting in one relevant study. This is purely practical measure pertaining to time feasibility of the experiments. In each optimisation run we perform fixed number of trials. We use 150 trials as baseline, adding 10-50 trials for more parametrised containers. The actual number of trials for a container is reflected in the study table 5.4. The optimisation objective is the maximum final fitness achieved across three runs with different random seeds. Each algorithm run optimises for 15 generations flat. Because we are dealing with stochastic method this number does not guarantee neither good or bad performance and has been arbitrarily chosen from experience with preliminary experiments.

5.2.1 Results

The hyper-parameter selection for each run is done within limits detailed in table tables 5.2 and 5.3. We present the limits in the form where we disclose minimum (Min) value the search engine will consider for the parameter, maximum considered value (Max) and recommended step that the search engine is meant to adhere to when tuning the value. The only exception is presentation of the method of crossover in ES, a categorical variable of two possible values. Not to break the patter we present the two values in the Min and Max columns arbitrarily.

The shorthand $arch.T$ denotes period of archiving in generations. W_0 is the starting value of W or W' in *increasing fitness* or *increasing novelty* respectively. Rate of decay is labelled r_D. Parameters α and γ for DE are respectively scaling factor and recombination rate. For ES we label mutation rate mr and crossover rate cr. By σ we mean scale of mutation. Parameters μ and λ are populations as described in section 3.1.1. Parameter *pop* is population of DE.

Running the optimisation we have obtained studies with the parameters described in table 5.4. The table features score of the best performing individual, total number of trials performed in the course of the study and number of trials achieving the best score.

While Optuna has internal mechanism for selecting the best trial, sometimes a larger quantity of trials achieve the best performance. This concerns mainly CartPole, where we have on several occasions discovered solving individual for multiple configurations.

When multiple best trials are available we will select the one which has the highest operation rates K where $K = CR + MR$ for ES and $K = \alpha + \gamma$ for DE. The logic behind this is that suppression of exploratory operators leads to greater exploitation of the roll on initial population. Therefore we assume less exploratory algorithm has a higher chance of simply being lucky on the tested

Parameter	CartPole Min	Max	Step	Lunar Lander Min	Max	Step
method	uniform	artithm.	-	uniform	artithm.	-
μ	10	30	10	40	70	10
λ	10	30	10	40	70	10
mr	0.00	0.10	0.01	0.00	0.50	0.01
cr	0.30	1.00	0.10	0.30	1.00	0.10
σ	0.50	2.00	0.50	0.50	3.00	0.50
arch. T	2	5	1	2	5	1
arch. batch	1	5	1	1	5	1
limit	200	300	10	-200	-50	10
W_0	0.20	0.70	0.10	0.20	1.00	0.10
r_D	0.50	5.00	0.50	0.50	5.00	0.50
p_U	0.10	0.90	0.10	0.10	0.90	0.10

Table 5.2: Hyper-parameters limits for Bayesian search ES

Parameter	CartPole Min	Max	Step	Lunar Lander Min	Max	Step
pop	5.00	20.00	5.00	40.00	70.00	10.00
α	0.00	1.00	0.10	0.00	1.00	0.10
γ	0.30	1.00	0.10	0.30	1.00	0.10
arch. T	2.00	5.00	1.00	2.00	5.00	1.00
arch. batch	1.00	5.00	1.00	1.00	5.00	1.00
limit	200.00	300.00	10.00	-200.00	-50.00	10.00
W_0	0.20	1.00	0.10	0.20	1.00	0.10
r_D	0.50	5.00	0.50	0.50	5.00	0.50

Table 5.3: Hyperparameters limits for Bayesian search DE

random seeds. If the tie is still not broken we select the random trial from the ones with lowest population score $S = \mu + \lambda$ for ES and $S =$ population size for DE. The final hyper-parameters found are presented in tables tables 5.5 to 5.8.

5.2.2 Discussion

We have obtained optimised hyper-parameter sets for each container. While the optimisation is stochastic and therefore not stable it allows us to explore relationships between the hyper-parameters more efficiently than straight grid search which would require several hundreds of runs with the same setting. Particularly interesting are the results of ES where the uniform method of crossover completely dominates both task, accompanied with quite skewed values of parameter p_U. We can assume one of the reasons for this dominance is the p_U is in our case adjustable to the needs of the objective handling method, while in arithmetic crossover the redistribution of values is fixed. We can also observe differences between the algorithms - DE requires much less population to function similarly. Also quite expectedly CartPole environment is less demanding on population than Lunar Lander where some containers have reached the preset limit. Interestingly while the mutation rates for ES are similar on both environments, BS on Lunar Lander generally utilised a lesser fraction of the allowed range. On the other hand it consistently produced greater values of σ. We can conclude that the Lunar Lander requires non-frequent but more impactful mutations.

environment	alg. container	Best Score DE	Best Score ES	Trials DE	Trials ES	Best Trials DE	Best Trials ES
CartPole	elite archive	500	500	150	200	20	42
	fit. archive	500	500	192	190	46	30
	fitness	500	500	100	100	31	15
	inc. fitness	500	500	160	160	71	27
	inc. novelty	500	500	160	160	62	43
	limit archive	500	500	150	175	33	27
	novetly archive	500	500	195	195	18	26
	pure novelty	500	500	150	150	76	22
Lunar Lander	elite archive	115	111	150	155	1	1
	fit. archive	92	105	160	160	41	1
	fitness	80	92	120	153	5	1
	inc. fitness	85	117	160	160	3	1
	inc. novelty	23	115	170	170	1	1
	limit archive	12	26	170	175	2	57
	novetly archive	-4	-57	160	160	1	1
	pure novelty	82	40	150	150	1	1

Table 5.4: Performed Optuna studies for each container - Best score = maximum achieved fitness, Trials = total trial count, Best trials = no. trials achieving the maximum fitness, DE and ES are names of the EAs

5.2.3 Incremental grid search

The goal of this experiment is refining the hyper-parameters obtained in the Bayesian search described in section 5.2. While the refinement is primarily concerned with performance we also take into account diversity of produced population as secondary measure. The algorithm of incremental grid search is described in section 4.4.3.

5.2.4 Results

To avoid duplication we present the results of the incremental grid search (IGS) in delta tables 5.9 to 5.12 derived for hyper-parameter i like so:

$$\Delta_i = F_i - O_i$$

where F_i is the value of hyper-parameter at the finish of the grid search and O_i is the original value passed from step 5.2. This way we can simply add contents of the delta table to the original in order to get the final outcome. We denote the hyper-parameters of ES as CR for crossover, MR for mutation rate, μ and λ correspond to the ES parameters of the same name. In DE we denote scaling factor as α, γ denotes recombination and population is denoted as pop.

	arch. batch	arch. T	cr	method	p_U	r_D	W_0	μ	limit	λ	mr	σ
fitness	-	-	1.00	U	0.9	-	-	10	-	30	0.09	0.50
pure novelty	-	-	1.00	U	0.3	-	-	10	-	30	0.06	0.50
inc. fitness	-	-	1.00	A	-	0.5	0.3	20	-	20	0.10	1.00
inc. novelty	-	-	1.00	U	0.7	1.0	0.4	10	-	20	0.09	1.00
fit. archive	1.0	5.0	1.00	A	-	-	-	20	-	20	0.09	2.00
elite archive	2.0	3.0	1.00	U	0.5	-	-	20	-	30	0.08	1.50
novetly archive	5.0	5.0	1.00	U	0.4	-	-	20	-	30	0.07	1.50
limit archive	5.0	4.0	1.00	U	0.6	-	-	20	220.0	30	0.06	1.00

Table 5.5: Bayesian Search results CartPole ES

	arch. batch	arch. T	cr	r_D	W_0	pop	limit	mr
fitness	-	-	1.00	-	-	15.00	-	0.50
pure novelty	-	-	1.00	-	-	20.00	-	0.80
inc. fitness	-	-	0.90	2.5	1.0	10.00	-	1.00
inc. novelty	-	-	1.00	3.5	0.4	10.00	-	0.70
fit. archive	3.0	3.0	0.90	-	-	20.00	-	0.90
elite archive	4.0	2.0	0.50	-	-	20.00	-	0.80
novetly archive	3.0	3.0	0.80	-	-	15.00	-	0.60
limit archive	3.0	3.0	1.00	-	-	20.00	240.0	0.90

Table 5.6: Bayesian Search results CartPole DE

5.2.5 Discussion

We can see that there was a room for improving almost all of the proposed hyper-parameter sets in all of the hyper-parameters subject to refinement. In some instances we sewe that the algorithm shrank the population related parameters such as μ or λ. This is likely result of the secondary diversity maximisation policy where the algorithm didn't find better performing solution but found one producing more diverse population. We assume this because lowering the population pool can only limit the exploitation of already generated individuals but it increases the diversity denser population implicitly lowers this statistic.

5.2.6 Generational grid search

The goal of this experiment is obtaining an optimised length of the search, given the refined hyper-parameters. Having obtained hyper-parameters in section 5.2.3 we now search for the optimal number of generation for which the evolution should go on. We perform the grid search as described in section 4.4.2.

It should be noted that to obtain the results we have not made a single long algorithm run with sampling of population on the specified generation but every generation count represents its own experimental run. While this strategy is significantly less efficient it is required as some objective handling methods, particularly combinations of novelty and fitness (3.2.3) have aspects that rely on total number of generation. We evaluate each generation count of the algorithm on 5 different seeds. We have searched over the following generation counts:

- For CartPole $g \in \{5, 10, 15, 20, 25, 30*, 40*, 50*\}$
- For Lunar Lander $g \in \{10, 15, 25, 35, 50, 60, 80\}$

	arch. batch	arch. T	cr	method	p_U	r_D	W_0	μ	limit	λ	mr	σ
fitness	-	-	1.00	U	-	-	-	40	-	40	0.02	1.50
pure novelty	-	-	0.70	U	0.2	-	-	70	-	40	0.06	2.50
inc. fitness	-	-	1.00	U	0.7	2.0	0.6	70	-	70	0.25	0.50
inc. novelty	-	-	0.50	U	0.7	1.5	0.9	70	-	50	0.28	1.50
fit. archive	4.0	2.0	1.00	U	0.6	-	-	50	-	70	0.01	2.50
elite archive	2.0	4.0	0.90	U	0.8	-	-	70	-	70	0.01	2.50
novetly archive	3.0	5.0	1.00	A	-	-	-	50	-	70	0.10	2.00
limit archive	4.0	5.0	0.50	U	0.5	-	-	70	-90.0	50	0.02	2.50

Table 5.7: Bayesian Search results for Lunar Lander ES

	arch. batch	arch. T	γ	r_D	W_0	pop	limit	α
fitness	-	-	0.30	-	-	40.00	-	0.06
pure novelty	-	-	0.40	-	-	40.00	-	0.15
inc. fitness	-	-	0.40	4.0	0.5	40.00	-	0.13
inc. novelty	-	-	0.60	1.0	0.4	70.00	-	0.04
fit. archive	2.0	2.0	0.60	-	-	40.00	-	0.30
elite archive	4.0	5.0	0.50	-	-	40.00	-	0.30
novetly archive	5.0	4.0	0.90	-	-	60.00	-	0
limit archive	1.0	2.0	0.80	-	-	50.00	-180.0	0.80

Table 5.8: Bayesian Search results for Lunar Lander DE

Because some containers have not been able to reach solving individual in CartPole in the original scope of the grid search and have exhibited interesting enough behaviour, we have extended their gridsearches. The extension generation counts have been marked with * symbol.

5.2.7 Results

We showcase the resulting patterns in figs. 5.1 and 5.2.

Figures 5.1 and 5.2 however have been pruned for readability and do not represent the full range of values that have been taken into account during the ultimate selection such as the population median or mean. The shaded area here represents quantiles q_{25} and q_{75} other values have been pruned. The actual scope of values considered for each contained is represented by chart that have been for readability sake put into section A.1.

The hand-selected generation counts for the final evaluation are presented in table 5.13.

5.2.8 Discussion

In CartPole environment we have been able to pinpoint hyper-parameter settings converging to solving individuals. In both algorithms (ES and DE), the fitness based methods have converged, however novelty is able to produced solving individuals mainly in DE. On the other hand increasing novelty converges in ES but not DE where it is surpassed by pure novelty. This might be linked ot the fact that in DE new individual always has to replace only one predetermined individual, making this process less reliable with the shifts in interpolation between two objectives. This difference in performance of increasing novelty is also

	Δ_{CR}	Δ_{μ}	Δ_{λ}	Δ_{MR}
pure novelty	-0.1	-	-	-
inc. fitness	-	+15.00	-	-
inc. novelty	+0.1	-	-15.00	+0.05
fit. archive	-	+30.00	+15.00	+0.1
elite archive	-0.1	-	+15.00	+0.1
novetly archive	-0.1	+15.00	+15.00	-
limit archive	0.1	-15.00	-	-

Table 5.9: Delta table of hyperparameters for ES on Lunar Lander after IGS step. Table shows difference in crossover rate, population, new generation pool and mutation rate

	Δ_{γ}	Δ_{pop}	Δ_{α}
pure novelty	+0.1	-10.0	-0.1
inc. fitness	-	+10.0	-
inc. novelty	+0.1	-	-
fit. archive	-0.1	-	-0.1
elite archive	+0.2	+10.0	-
novetly archive	-	-	+0.1
limit archive	-	+10.0	+0.1

Table 5.10: Delta table of hyper-parameters for DE on Lunar Lander environment after IGS step. Table shows difference in recombination rate, population and scaling factor.

replicated in Lunar Lander environment, it is therefore unlikely simply result of bad hyper-parameter optimisation.

In Lunar Lander we can again see domination of fitness base methods. Interestingly for ES increasing fitness dominates pure fitness. We can infer from the stability of the fitness results that the fitness search has reached false optimum that the archival versions and increasing fitness have been able to break out of. These three methods are able to reliably achieve well behaved individuals, almost very-well behaved individuals. However we never reach true solution with either. Replays of successful individuals produced by these methods show tendency to stop the descend right above the landing pad in a hover. This could be caused by several factors. First, it might be the skill to land, requiring controlled shutdown of engines, is substantially different from the successful navigation in the sky, where slowing of descend is imperative and we have reached stochastic bottleneck too narrow for our search and agent architecture. Second, we have selected the discrete version of the environment so since the agent is unaware of the previous state the shuttle cannot learn to gradually suspend the engines but is stuck at last position that resolves to engine fire. Third factor is that we have limited length of one episode to 120 steps, possibly eliminating solutions that would ultimately achieve better performance on account of being too slow. While it seems unlikely that this would be the only factor it could certainly be contributing to the stalemate.

	Δ_γ	Δpop	$\Delta\alpha$
fitness	-	+5.000	-
novelty	0.200	+10.000	-0.100
inc. fitness	-0.100	+10.000	-0.100
inc. novelty	-	+5.000	-
fit. archive	-0.100	-	-0.200
elite archive	-	+5.000	+0.200
novetly archive	-0.100	-5.000	-0.100
limit archive	-0.100	+10.000	-0.100

Table 5.11: Delta table of hyper-parameters for DE on CartPole environment after IGS step. Table shows difference in recombination rate, population and scaling factor.

	ΔCR	$\Delta\mu$	$\Delta\lambda$	ΔMR
fitness	-0.100	+5	+5	-
novelty	-	+5	-5	-0.050
inc. novelty	-	+10	-10	-
fit. archive	-0.100	-5	+10	-
elite archive	-0.100	-	-	0.000
novetly archive	-0.100	+5	+5	-0.050
limit archive	-	+5	-	-0.050

Table 5.12: Delta table of hyper-parameters for ES on CartPole environment after IGS step. Table shows difference in crossover rate, population, new generation pool and mutation rate

5.3 Final Experiment

In the previous experiments we have conducted, we had to limit the variety of initial conditions represented by random seed on which they were evaluated. The goal of this experiment is to obtain data on large variety of initial condition with the optimised hyper-parameters to perform comprehensive analysis of the relationships between utilised containers.

To do so we have run the parameterised algorithms on 45 distinct seeds for each environment from which we collected data about the last population. The data is plotted in figs. 5.3 and 5.4. We have subjected the collect data to the statistical tests described in section 4.5. We will now go over the application methods. First we will group results of both evolutionary algorithms we have utilised (DE and ES) collected across all containers on each of the environments. Then we will analyse groups based on objective used, following the internal divide in section 3.2. We will analyse both internal consistency of the groups and their relationships. Then we will analyse differences between the objective handling methods granularly. Since the groupings of results are different in each case, we claim them to be different test families for the purposes of adjustments to *FWER*. We will implicitly use the significance threshold $\alpha = 0.05$ as is common in publications.

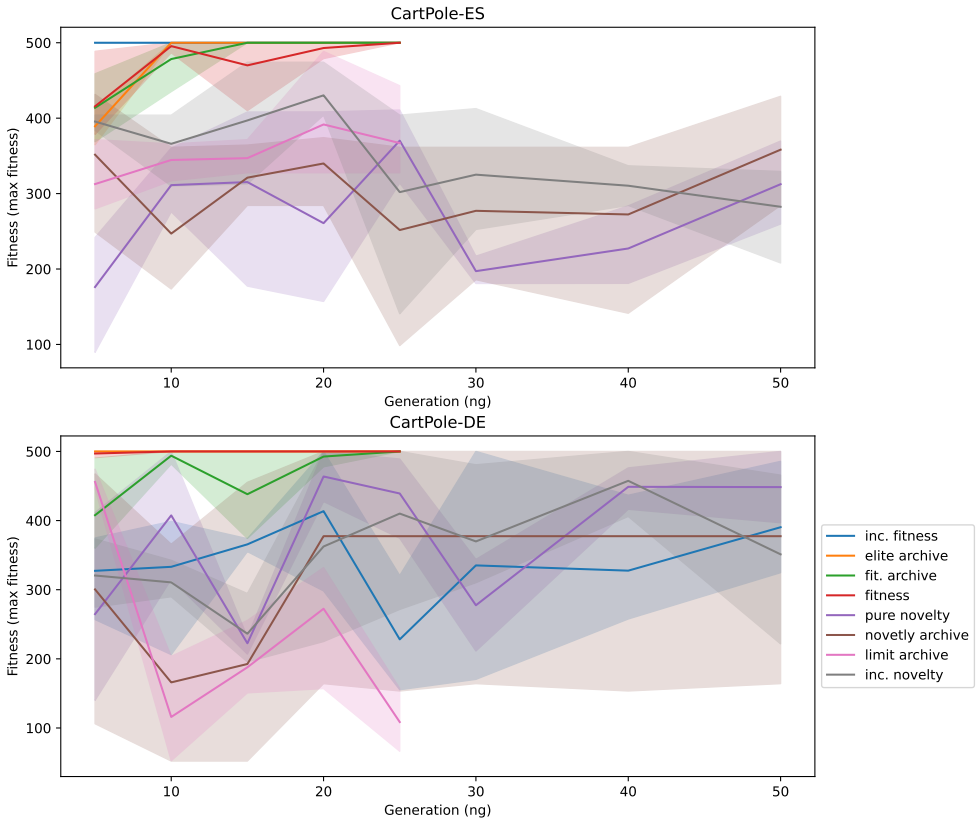

Figure 5.1: Fitness/Generation Characteristics for CartPole environment (ES up, DE down)

5.3.1 EA comparison

Since we examine only two distinct EA implementations we can directly perform Wilcoxon test. For comparability reasons we analyse EAs performed on each of the environments separately. On each environment the Wilcoxon test rejects the null hypothesis with p values $p_{LL} = 3.11 \times 10^{-12}$ and $p_{CT} = 0.0002$ for Lunar lander and CartPole respectively. On each of the environments we calculate the direction of the difference through Hodges–Lehmann estimator (section 4.5.2) over the differences $d_i = ES_i - DE_i$. Directional values are $HL_{LL} = -54.22$ and $HL_{CT} = 29.12$ suggesting that DE has been more performant on Lunar lander while ES has been more performant on CartPole environment.

We also investigate the diversity in the final populations produced by our algorithms. On CartPole the H_0 is supported with $p_{CT} = 0.49$ but for Lunar Lander environment the difference is supported $p_{LL} < 0.0001$, rejecting H_0. The difference skews slightly for the differential evolution $HL_{LL} = -0.122$.

5.3.2 Group comparison analysis

We would like to compare objective methods as such (fitness, novelty, combination of both). To do so we have grouped the objective handling methods according to the division in section 3.2.

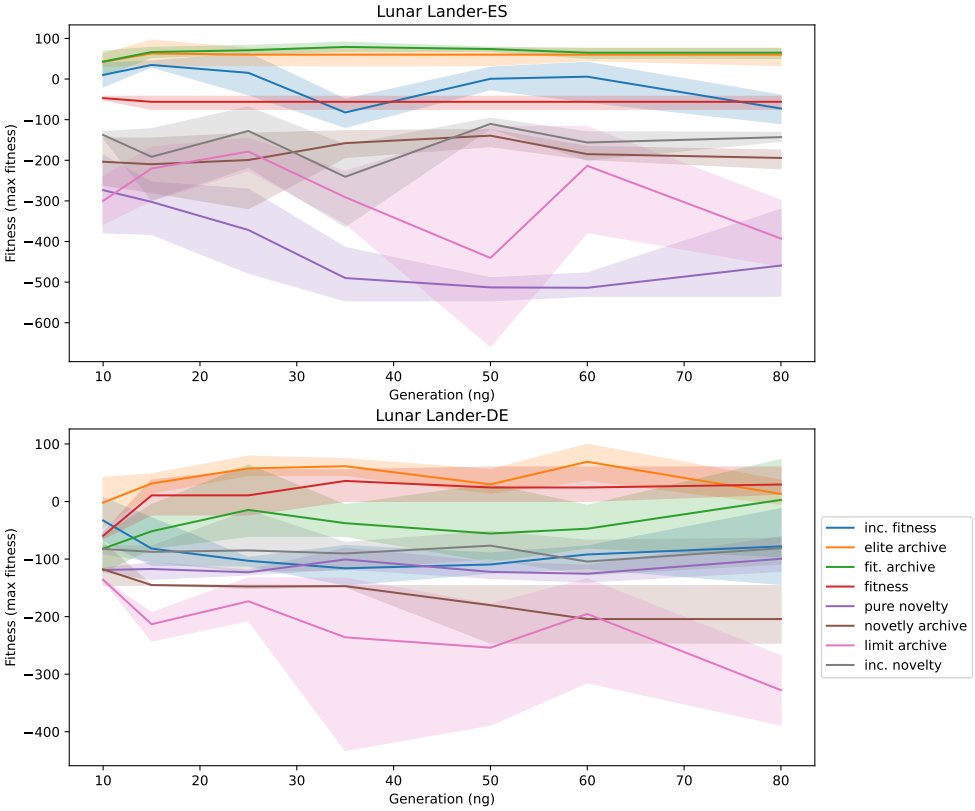

Figure 5.2: Fitness/Generation Characteristics for Lunar Lander environment (ES up, DE down)

Internal difference analysis

We want to assess the group internal differences by the means of Friedman test for groups of 3 and Wilcoxon for the group of 2. We do this separately for each environment. The results are presented in the tables 5.14 and 5.15. p_{DE} denotes p-value for test only on differential evolution runs. Analogically with p_{ES}. Testing the aggregate of both algorithms we get p_{Total}.

P-values rejecting the null hypotheses are highlighted. We can see that in CartPole differential evolution the two methods of combination fitness and novelty are behaving so similarly, we do not have enough evidence to reject assumption that they might be behaving the same. This is even more pronounced in Lunar Lander but only in terms of produced diversity. The fitness group does not produce enough evidence of internal differences only in Lunar Lander and only in aggregate. It should be noted that while we are using academic standard $\alpha = 0.05$ it is not uncommon to use lower standard $\alpha = 0.1$ which would allow us to reject H_0 in several of the aforementioned cases.

Intergroup difference analysis

Having assessed how the groups behave internally, we now compare them between each other on both of the environments with Wilcoxon test. When pairing the performance we pair median value of the values achieved by the group members on a particular initial seed using a particular algorithm.

	Lunar Lander		CartPole	
	DE	ES	DE	ES
fitness	35	25	15	10
pure novelty	15	10	25	40
fit. archive	80	15	35	25
inc. fitness	15	15	15	20
inc. novelty	25	25	20	40
novetly archive	25	60	20	20
elite archive	60	15	20	10
limit archive	25	25	20	20

Table 5.13: Selection of generations hyper-parameter based on performed grid search

group	test	Lunar Lander			CartPole		
group name	test	p_{Total}	p_{DE}	p_{ES}	p_{Total}	p_{DE}	p_{ES}
novelty obj.	friedman	$\mathbf{6.192 \times 10^{-9}}$	$\mathbf{7.860 \times 10^{-11}}$	$\mathbf{2.569 \times 10^{-4}}$	**0.012**	$\mathbf{1.424 \times 10^{-8}}$	$\mathbf{8.648 \times 10^{-5}}$
fitness obj.	friedman	0.096	**0.008**	**0.022**	**0.030**	**0.045**	**0.019**
combination obj.	wilcoxon	$\mathbf{6.401 \times 10^{-5}}$	**0.009**	$\mathbf{4.944 \times 10^{-10}}$	$\mathbf{2.739 \times 10^{-6}}$	0.550	$\mathbf{8.402 \times 10^{-8}}$

Table 5.14: Fitness based tests of group difference

The results are presented in tables 5.16 and 5.17 for Lunar Lander environment and in tables 5.18 and 5.19 for CartPole environment.

Performance wise the *fitness group* dominates both environments. We can see that *combinations* dominate over *novelty group*. In diversity we see that on the level of individual EAs the differences are statistically significant but in aggregate test the differences may cancel out.

5.3.3 Objective handling analysis

Now we want to compare the objective handling methods based on fitness of individuals they have produced. Friedman test rejects the null hypothesis for both Lunar Lander ($\chi_F = 328.39$, $p_F = 5.1 \times 10^{-67}$) and CartPole ($\chi_F = 326.66$, $p_F = 1.21 \times 10^{-66}$). So we proceed to the pairwise Wilcoxon tests on both environments to obtain significance of their differences.

The full results for fitness are shown in tables 5.22 and 5.23 respectively. For the cases where the difference wasn't proven significant we do not report the calculated direction. We summarise the results of the comparisons in tables 5.20 and 5.21. The results are summarised in format:

#of won comparisons/#of lost comparisons/#of ties

. We count tie as either statistically insignificant difference or significant difference with value of Hodges-Lehman estimator $HL = 0$.(section 4.5.2)

We can see that for both environments the pure fitness method dominates with fitness archives being roughly on the same footing. Notably pure novelty mostly dominates its own archival variants.

Elite archive does not show strong dominance over random *fitness archive*. Even in CartPole where the null hypotheses of their relationship was rejected, the direction came out 0 suggesting the difference to be purely about distribution not systematic shift between the distributions.

group group name	test test	Lunar Lander p_{Total}	 p_{DE}	 p_{ES}	CartPole p_{Total}	 p_{DE}	 p_{ES}
novelty obj.	friedman	$\mathbf{7.385 \times 10^{-17}}$	$\mathbf{1.928 \times 10^{-9}}$	$\mathbf{3.454 \times 10^{-11}}$	$\mathbf{6.745 \times 10^{-16}}$	$\mathbf{2.723 \times 10^{-13}}$	$\mathbf{6.773 \times 10^{-5}}$
fitness obj.	friedman	**0.036**	$\mathbf{1.004 \times 10^{-10}}$	$\mathbf{2.398 \times 10^{-4}}$	$\mathbf{1.802 \times 10^{-13}}$	$\mathbf{6.938 \times 10^{-10}}$	$\mathbf{1.935 \times 10^{-6}}$
combination obj.	wilcoxon	0.101	0.712	0.088	$\mathbf{6.909 \times 10^{-6}}$	0.082	$\mathbf{5.333 \times 10^{-6}}$

Table 5.15: Diversity based tests of group difference

A	B	HL_{Total}	p_{Total}	HL_{DE}	p_{DE}	HL_{ES}	p_{ES}
novelty obj.	fitness obj.	-175.50	$\mathbf{5.410 \times 10^{-16}}$	-151.06	$\mathbf{1.705 \times 10^{-13}}$	-207.97	$\mathbf{3.411 \times 10^{-13}}$
novelty obj.	combination obj.	-93.14	$\mathbf{6.634 \times 10^{-13}}$	-45.94	$\mathbf{3.262 \times 10^{-10}}$	-147.10	$\mathbf{3.173 \times 10^{-9}}$
fitness obj.	combination obj.	80.45	$\mathbf{5.251 \times 10^{-12}}$	94.26	$\mathbf{5.684 \times 10^{-13}}$	62.18	$\mathbf{1.187 \times 10^{-4}}$

Table 5.16: Fitness based Wilcoxon tests of between objective groups on Lunar Lander environment. HL_x denotes calculated direction of the difference (A-B) for algorithm x, p_x is p value of the test for algorithm x. *Total* denotes collection of comparisons from both ES and DE

A	B	HL_{Total}	p_{Total}	HL_{DE}	p_{DE}	HL_{ES}	p_{ES}
novelty obj.	fitness obj.	0.09061	$\mathbf{1.744 \times 10^{-11}}$	0.05	$\mathbf{1.471 \times 10^{-4}}$	0.14	$\mathbf{1.091 \times 10^{-10}}$
novelty obj.	combination obj.	-	0.071	-0.06	$\mathbf{2.306 \times 10^{-7}}$	0.03	**0.036**
fitness obj.	combination obj.	-0.101976	$\mathbf{3.043 \times 10^{-15}}$	-0.10	$\mathbf{1.705 \times 10^{-13}}$	-0.10	$\mathbf{6.780 \times 10^{-8}}$

Table 5.17: Diversity based Wilcoxon tests of between objective groups on Lunar Lander environment. HL_x denotes calculated direction of the difference (A-B) for algorithm x, p_x is p value of the test for algorithm x. *Total* denotes collection of comparisons from both ES and DE

Novelty archive dominates *limit archive* in Lunar Lander environment. In CartPole *limit archive* dominates *novelty archive*.

Diversity Comparison Results

Now that we have made the comparison on achieved fitness we want to compare the algorithms also based on the produced diversity. The Friedman test rejects the null hypothesis also for diversities both for Lunar Lander ($\chi_F = 192.23$, $p_F = 5 \times 10^{-38}$) and CartPole ($\chi_F = 163.918$, $p_F = 4.8 \times 10^{-32}$) and so we can safely perform Wilcoxon test on the combinations of objective handling methods. Results are shown in tables 5.24 and 5.25.

A	B	HL_{Total}	p_{Total}	HL_{DE}	p_{DE}	HL_{ES}	p_{ES}
novelty obj.	fitness obj.	-240.58	$\mathbf{1.119 \times 10^{-15}}$	-294.50	$\mathbf{3.359 \times 10^{-8}}$	-195.25	$\mathbf{1.705 \times 10^{-13}}$
novelty obj.	combination obj.	-91.13	$\mathbf{3.697 \times 10^{-7}}$	-103.54	$\mathbf{9.014 \times 10^{-4}}$	-73.12	$\mathbf{1.041 \times 10^{-5}}$
fitness obj.	combination obj.	150.33	$\mathbf{9.573 \times 10^{-15}}$	185.12	$\mathbf{3.359 \times 10^{-8}}$	114.88	$\mathbf{1.934 \times 10^{-7}}$

Table 5.18: Fitness based Wilcoxon tests of between objective groups on Cart-Pole environment. HL_x denotes calculated direction of the difference (A-B) for algorithm x, p_x is p value of the test for algorithm x. *Total* denotes collection of comparisons from both ES and DE

A	B	HL_{Total}	p_{Total}	HL_{DE}	p_{DE}	HL_{ES}	p_{ES}
novelty obj.	fitness obj.	-	0.758	-0.12	**0.002**	0.08	**0.001**
novelty obj.	combination obj.	-	0.359	-0.25	$\mathbf{8.622 \times 10^{-10}}$	0.15	$\mathbf{3.505 \times 10^{-6}}$
fitness obj.	combination obj.	-	0.107	-0.12	$\mathbf{5.479 \times 10^{-10}}$	0.07	**0.006**

Table 5.19: Diversity based Wilcoxon tests of between objective groups on Cart-Pole environment. HL_x denotes calculated direction of the difference (A-B) for algorithm x, p_x is p value of the test for algorithm x. *Total* denotes collection of comparisons from both ES and DE

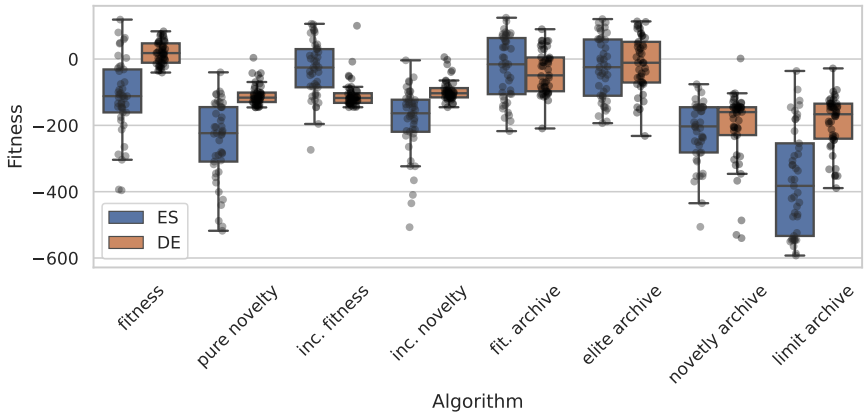

Figure 5.3: Plot of results of final experiment in Lunar Lander environment.

5.3.4 Final Experiment Discussion

We have found out that each EA dominates different task in performance. This is not completely unexpected as the ES in our case is quite limited with the static sigma and therefore cannot adjust to the needs of the population during its run on a more complex task (Lunar Lander). On the other hand we get this adjustment automatically with DE algorithm without specifying any hyper-parameter. From the performance chart in figs. 5.3 and 5.4 we can see that the performance difference may be mostly attributable to the spread of results rather than performance at the top. This is supported by the fact that in Lunar Lander task the differential evolution clearly produced more diverse populations than ES.

As stated before the fitness objective handling methods dominate the tasks. From tables 5.14 and 5.15 we can also see that their results are also generally more homogenous than the other groups (evidenced by mostly greater p-values). The combinations of fitness and novelty perform somewhat better than the methods

	Fit. Sum.	Fit. DE	Fit. ES	Div. Sum.	Div. DE	Div. ES
fit. archive	5/0/2	5/1/1	5/0/2	0/4/3	1/4/2	0/5/2
elite archive	5/0/2	5/0/2	5/0/2	0/5/2	0/7/0	1/3/3
fitness	4/0/3	6/0/1	3/3/1	1/4/2	3/3/1	0/4/3
inc. fitness	4/2/1	2/3/2	5/0/2	4/1/2	5/0/2	3/2/2
inc. novelty	3/4/0	2/3/2	1/3/3	4/0/3	5/0/2	4/1/2
pure novelty	2/5/0	2/3/2	1/4/2	6/0/1	5/0/2	5/0/2
novetly archive	1/6/0	0/6/1	1/4/2	4/1/2	1/3/3	6/0/1
limit archive	0/7/0	0/6/1	0/7/0	0/4/3	1/4/2	0/4/3

Table 5.20: Wilcoxon performance comparison summary fo Lunar Lander env. Comparison score - *#wins/#loses/#ties*. Fit. - fitness comparisons on the EA; Div. - diversity comparisons on EA. Sum. = combination of both EAs

	Fit. Sum.	Fit. DE	Fit. ES	Div. Sum.	Div. DE	Div. ES
fitness	5/0/2	5/0/2	4/1/2	5/1/1	4/1/2	2/1/4
fit. archive	5/0/2	5/0/2	4/1/2	2/3/2	2/4/1	2/1/4
elite archive	5/0/2	5/0/2	7/0/0	0/5/2	1/4/2	0/6/1
inc. fitness	4/3/0	2/3/2	4/1/2	2/3/2	4/0/3	0/6/1
pure novelty	1/4/2	3/3/1	0/5/2	6/0/1	4/0/3	6/0/1
inc. novelty	1/4/2	2/4/1	0/5/2	5/0/2	5/0/2	2/0/5
limit archive	1/4/2	0/6/1	3/4/0	0/3/4	1/5/1	2/1/4
novetly archive	0/7/0	0/6/1	0/5/2	0/5/2	0/7/0	2/1/4

Table 5.21: Wilcoxon performance comparison summary fo CartPole env.Comparison score - *#wins/#loses/#ties*. Fit. - fitness comparisons on the EA; Div. - diversity comparisons on EA. Sum. = combination of both EAs

relying on novelty only.

The per-objective-handling-method Wilcoxon tests confirm this with some similarities uncovered between pure novelty and increasing novelty. This is not completely unexpected as the increasing emphasis of novelty diverges the population into similar randomised conditions.

We have also confirmed observation from section 5.2.6 that increasing fitness dominates pure fitness in Lunar Lander using ES. On fig. 5.3 we can however see that the domination is mainly distributional. this would suggest that pure fitness method is able to break out of the false optima but the increasing fitness is just more reliable in the process.

Archiving provides great benefit to the fitness methods in ES algorithm on Lunar Lander environment. There it improved on the pure fitness performance. The elitist and random solution are barely distinguishable with Wilcoxon null hypothesis being often sustained in the comparison or direction being of negligible magnitude. On the other hand pure novelty almost exclusively dominates its archival variants both in raw performance and diversity. This suggests a trend when archives in novelty do provide a performance benefit they do so by stabilising the population rather than adding more variety to it. Unfortunately in the case of our setup this did not provide the any performance boost.

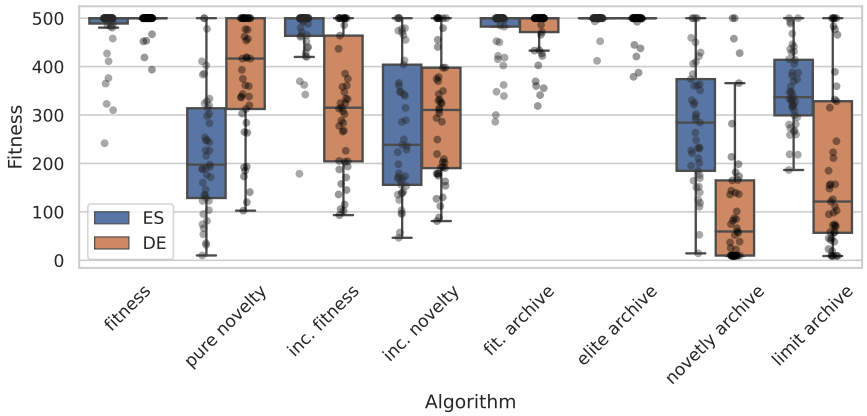

Figure 5.4: Plot of results of final experiment in CartPole environment.

Limit archive behaves poorly in Lunar Lander but better than random novelty archive in CartPole. We can see in figs. 5.3 and 5.4 that the difference is mainly in distributions of performances rather than achieving strictly better solutions. Therefore we propose that the difference lies in how difficult is it to reach the respective limits within the environment. Since the limits were optimised for, it suggests that simply choosing lower limits does not significantly improve on the performance as lesser quality individuals will now be periodically reinserted back into the population. Observing relative success of increasing fitness method we propose shifting threshold would be a better solution.

5.4 Summary Discussion

We have 16 container identified by the combination of objective handling method and EA used. For all of these 16 containers we have found and refined a set of hyper-parameters used in the final set of experiments. We have identified existence of false optima in Lunar Lander and delineated possible reasons why our evolutionary optimisation has not been able to completely break out of it. We have performed comparative tests on the EAs in aggregate and objective handling methods grouped by underlying objective. We have uncovered some differences in performances of the algorithms, mainly that DE produces slightly more diverse population in Lunar Lander and is more performant on the environment. We attribute it to the implicit dynamics of the DE. On the other hand, we are utilising quite static version of ES and propose that more dynamic ES algorithm might be more performant on the Lunar Lander environment as it might be more prepared to tackle the deceptiveness of it. We have also found that fitness based objective handling method dominate in performance across the board. Novelty does not achieve better results even in the more deceptive of the two investigated environments, on the contrary novelty based methods were the weakest performance wise. However it can achieve more diverse populations. We propose this is due to the structure of the environments as CartPole never changes the core of the skill utilised for achieving maximum reward while Lunar Lander requires substantial switch in behaviour but only at the very end. Therefore in both cases the behavioural space is not compartmentalised enough to trap the novelty exploration

A	B	HL_{Total}	p_{Total}	HL_{DE}	p_{DE}	HL_{ES}	p_{ES}
fitness	pure novelty	131.676947	$\mathbf{2.004 \times 10^{-11}}$	127.795874	$\mathbf{3.070 \times 10^{-12}}$	134.345431	$\mathbf{1.119 \times 10^{-4}}$
fitness	inc. fitness	-	0.240	128.532971	$\mathbf{6.094 \times 10^{-10}}$	-74.627906	**0.005**
fitness	inc. novelty	93.049699	$\mathbf{4.302 \times 10^{-9}}$	114.128641	$\mathbf{3.070 \times 10^{-12}}$	-	0.069
fitness	fit. archive	-	0.861	54.150964	$\mathbf{4.266 \times 10^{-5}}$	-83.616144	**0.011**
fitness	elite archive	-	0.360	-	0.231	-88.211462	**0.007**
fitness	novetly archive	168.466936	$\mathbf{1.409 \times 10^{-11}}$	207.896223	$\mathbf{3.070 \times 10^{-12}}$	110.273594	$\mathbf{6.528 \times 10^{-4}}$
fitness	limit archive	235.629767	$\mathbf{2.223 \times 10^{-13}}$	204.199313	$\mathbf{1.592 \times 10^{-12}}$	268.610349	$\mathbf{2.856 \times 10^{-8}}$
pure novelty	inc. fitness	-92.98606	$\mathbf{5.518 \times 10^{-7}}$	-	1.000	-208.065336	$\mathbf{4.188 \times 10^{-10}}$
pure novelty	inc. novelty	-28.248713	**0.012**	-	0.231	-	0.075
pure novelty	fit. archive	-128.184049	$\mathbf{8.741 \times 10^{-12}}$	-70.965488	$\mathbf{8.783 \times 10^{-7}}$	-216.946715	$\mathbf{9.053 \times 10^{-9}}$
pure novelty	elite archive	-146.481877	$\mathbf{1.063 \times 10^{-12}}$	-100.578474	$\mathbf{4.508 \times 10^{-7}}$	-212.219335	$\mathbf{9.962 \times 10^{-10}}$
pure novelty	novetly archive	36.000119	**0.015**	77.254385	$\mathbf{2.242 \times 10^{-8}}$	-	1.000
pure novelty	limit archive	97.498708	$\mathbf{5.715 \times 10^{-8}}$	77.441572	$\mathbf{1.977 \times 10^{-7}}$	125.560413	$\mathbf{6.528 \times 10^{-4}}$
inc. fitness	inc. novelty	54.464739	$\mathbf{6.401 \times 10^{-4}}$	-	0.053	150.662294	$\mathbf{9.393 \times 10^{-9}}$
inc. fitness	fit. archive	-43.212918	$\mathbf{6.486 \times 10^{-4}}$	-69.290076	$\mathbf{1.615 \times 10^{-6}}$	-	1.000
inc. fitness	elite archive	-51.222516	**0.001**	-97.50726	$\mathbf{1.080 \times 10^{-7}}$	-	1.000
inc. fitness	novetly archive	138.320717	$\mathbf{7.926 \times 10^{-13}}$	79.183149	$\mathbf{1.080 \times 10^{-7}}$	186.831027	$\mathbf{1.577 \times 10^{-9}}$
inc. fitness	limit archive	200.763637	$\mathbf{3.655 \times 10^{-14}}$	77.426754	$\mathbf{1.960 \times 10^{-8}}$	338.863942	$\mathbf{1.295 \times 10^{-7}}$
inc. novelty	fit. archive	-96.897141	$\mathbf{9.969 \times 10^{-10}}$	-54.684347	$\mathbf{4.266 \times 10^{-5}}$	-153.997883	$\mathbf{4.941 \times 10^{-7}}$
inc. novelty	elite archive	-111.446886	$\mathbf{4.632 \times 10^{-12}}$	-83.858434	$\mathbf{5.516 \times 10^{-7}}$	-147.932789	$\mathbf{9.053 \times 10^{-9}}$
inc. novelty	novetly archive	67.581635	$\mathbf{2.382 \times 10^{-6}}$	89.305449	$\mathbf{9.393 \times 10^{-9}}$	-	0.201
inc. novelty	limit archive	132.155841	$\mathbf{1.435 \times 10^{-10}}$	88.211582	$\mathbf{2.102 \times 10^{-8}}$	189.355484	$\mathbf{5.771 \times 10^{-6}}$
fit. archive	elite archive	-	0.582	-	0.231	-	1.000
fit. archive	novetly archive	174.285356	$\mathbf{3.655 \times 10^{-14}}$	156.265235	$\mathbf{1.376 \times 10^{-10}}$	195.705804	$\mathbf{3.595 \times 10^{-10}}$
fit. archive	limit archive	242.093519	$\mathbf{1.409 \times 10^{-14}}$	151.852576	$\mathbf{2.017 \times 10^{-10}}$	348.97078	$\mathbf{7.958 \times 10^{-12}}$
elite archive	novetly archive	181.016289	$\mathbf{8.320 \times 10^{-15}}$	179.845674	$\mathbf{9.152 \times 10^{-12}}$	182.623177	$\mathbf{1.478 \times 10^{-11}}$
elite archive	limit archive	253.778388	$\mathbf{6.829 \times 10^{-15}}$	174.962109	$\mathbf{3.070 \times 10^{-12}}$	348.347918	$\mathbf{1.074 \times 10^{-11}}$
novetly archive	limit archive	62.807367	**0.011**	-	1.000	153.728544	$\mathbf{1.119 \times 10^{-4}}$

Table 5.22: Fitness Wilcoxon test over objectives handlers for Lunar Lander (HL_x - direction on algorithm x, p_x - p-value on algorithm x)

in better performing regions. In the case of Lunar Lander we are searching over position on x axis and average velocity of descend. While these are instructive features we can see how only a small fraction of the space would correspond to high achieving individuals with no reason for the search to prefer individuals from such enclave. This effect could be mitigated by automated behavioural space selection which may more effectively consider features of the individual alongside the raw observations but application of such algorithm was outside of the scope this thesis. [Rakicevic et al., 2020]

A	B	HL_{Total}	p_{Total}	HL_{DE}	p_{DE}	HL_{ES}	p_{ES}
fitness	pure novelty	179.666667	$\mathbf{6.542 \times 10^{-12}}$	100.75	$\mathbf{3.868 \times 10^{-5}}$	260.333333	$\mathbf{5.716 \times 10^{-7}}$
fitness	inc. fitness	79.916667	$\mathbf{2.858 \times 10^{-6}}$	171.666667	$\mathbf{4.424 \times 10^{-6}}$	-	1.000
fitness	inc. novelty	195.416667	$\mathbf{8.411 \times 10^{-13}}$	189.75	$\mathbf{1.471 \times 10^{-6}}$	198.833333	$\mathbf{1.869 \times 10^{-6}}$
fitness	fit. archive	-	0.444	-	0.141	-	1.000
fitness	elite archive	-	0.438	-	0.960	-0.5	**0.045**
fitness	novetly archive	287.5	$\mathbf{4.660 \times 10^{-14}}$	410.416667	$\mathbf{2.893 \times 10^{-7}}$	190.666667	$\mathbf{6.616 \times 10^{-7}}$
fitness	limit archive	212.666667	$\mathbf{4.471 \times 10^{-13}}$	316.583333	$\mathbf{6.657 \times 10^{-7}}$	126.75	$\mathbf{3.532 \times 10^{-6}}$
pure novelty	inc. fitness	-97.25	**0.002**	-	0.138	-253.833333	$\mathbf{3.301 \times 10^{-7}}$
pure novelty	inc. novelty	-	0.687	74.75	**0.002**	-	0.285
pure novelty	fit. archive	-168.083333	$\mathbf{3.433 \times 10^{-11}}$	-80.916667	**0.002**	-252.25	$\mathbf{3.136 \times 10^{-7}}$
pure novelty	elite archive	-190.083333	$\mathbf{2.480 \times 10^{-12}}$	-100.083333	$\mathbf{5.975 \times 10^{-5}}$	-279.5	$\mathbf{3.136 \times 10^{-7}}$
pure novelty	novetly archive	108.416667	**0.003**	274.416667	$\mathbf{2.007 \times 10^{-9}}$	-	0.285
pure novelty	limit archive	-	0.687	200.833333	$\mathbf{2.433 \times 10^{-5}}$	-130.583333	$\mathbf{1.113 \times 10^{-5}}$
inc. fitness	inc. novelty	110.5	$\mathbf{3.287 \times 10^{-5}}$	-	0.960	191.5	$\mathbf{1.344 \times 10^{-6}}$
inc. fitness	fit. archive	-63.416667	$\mathbf{9.251 \times 10^{-5}}$	-155.416667	$\mathbf{2.570 \times 10^{-5}}$	-	1.000
inc. fitness	elite archive	-86.083333	$\mathbf{4.789 \times 10^{-9}}$	-171.083333	$\mathbf{3.237 \times 10^{-6}}$	-9.75	**0.002**
inc. fitness	novetly archive	200.333333	$\mathbf{1.522 \times 10^{-12}}$	217.25	$\mathbf{2.169 \times 10^{-7}}$	188.333333	$\mathbf{8.534 \times 10^{-7}}$
inc. fitness	limit archive	128.083333	$\mathbf{3.061 \times 10^{-8}}$	136.166667	**0.002**	121.666667	$\mathbf{5.313 \times 10^{-6}}$
inc. novelty	fit. archive	-182.5	$\mathbf{1.214 \times 10^{-12}}$	-168.5	$\mathbf{2.299 \times 10^{-6}}$	-195.583333	$\mathbf{1.237 \times 10^{-6}}$
inc. novelty	elite archive	-204.333333	$\mathbf{3.442 \times 10^{-13}}$	-187.833333	$\mathbf{1.630 \times 10^{-6}}$	-217.666667	$\mathbf{5.577 \times 10^{-7}}$
inc. novelty	novetly archive	87.416667	**0.003**	182.833333	$\mathbf{3.497 \times 10^{-7}}$	-	1.000
inc. novelty	limit archive	-	0.687	121.666667	**0.021**	-78.75	**0.023**
fit. archive	elite archive	0.0	**0.006**	-	0.230	-1.416667	**0.034**
fit. archive	novetly archive	276.083333	$\mathbf{5.750 \times 10^{-14}}$	383.75	$\mathbf{2.893 \times 10^{-7}}$	187.416667	$\mathbf{8.534 \times 10^{-7}}$
fit. archive	limit archive	198.5	$\mathbf{3.545 \times 10^{-13}}$	298.583333	$\mathbf{7.852 \times 10^{-7}}$	117.25	$\mathbf{6.848 \times 10^{-7}}$
elite archive	novetly archive	302.833333	$\mathbf{3.903 \times 10^{-14}}$	403.583333	$\mathbf{2.893 \times 10^{-7}}$	215.666667	$\mathbf{5.281 \times 10^{-7}}$
elite archive	limit archive	220.833333	$\mathbf{7.593 \times 10^{-14}}$	315.916667	$\mathbf{7.102 \times 10^{-7}}$	145.416667	$\mathbf{3.136 \times 10^{-7}}$
novetly archive	limit archive	-68.416667	**0.003**	-	0.138	-71.75	**0.036**

Table 5.23: Fitness Wilcoxon test over objectives handlers for CartPole (HL_x - direction on algorithm x, p_x - p-value on algorithm x)

A	B	HL_{Total}	p_{Total}	HL_{DE}	p_{DE}	HL_{ES}	p_{ES}
fitness	pure novelty	-0.042555	**0.036**	-	1.000	-0.100771	**0.018**
fitness	inc. fitness	0.090809	$\mathbf{1.331 \times 10^{-4}}$	-	1.000	0.198355	$\mathbf{2.773 \times 10^{-7}}$
fitness	inc. novelty	-	0.505	-0.04303	**0.046**	-	1.000
fitness	fit. archive	0.073376	**0.006**	0.149382	$\mathbf{1.206 \times 10^{-5}}$	-	1.000
fitness	elite archive	0.217708	$\mathbf{2.037 \times 10^{-12}}$	0.228213	$\mathbf{2.306 \times 10^{-10}}$	0.200677	$\mathbf{1.753 \times 10^{-6}}$
fitness	novetly archive	0.225698	$\mathbf{3.939 \times 10^{-7}}$	0.460536	$\mathbf{1.251 \times 10^{-10}}$	-	1.000
fitness	limit archive	0.146048	$\mathbf{1.700 \times 10^{-5}}$	0.281818	$\mathbf{2.306 \times 10^{-10}}$	-	1.000
pure novelty	inc. fitness	0.149859	$\mathbf{2.568 \times 10^{-6}}$	-	1.000	0.317542	$\mathbf{1.994 \times 10^{-8}}$
pure novelty	inc. novelty	-	1.000	-	0.182	-	0.351
pure novelty	fit. archive	0.129778	$\mathbf{4.305 \times 10^{-9}}$	0.144914	$\mathbf{2.803 \times 10^{-7}}$	0.1164	$\mathbf{5.134 \times 10^{-4}}$
pure novelty	elite archive	0.262882	$\mathbf{2.182 \times 10^{-14}}$	0.234933	$\mathbf{2.916 \times 10^{-11}}$	0.293846	$\mathbf{4.027 \times 10^{-10}}$
pure novelty	novetly archive	0.272268	$\mathbf{2.288 \times 10^{-11}}$	0.456023	$\mathbf{3.695 \times 10^{-11}}$	0.106029	**0.002**
pure novelty	limit archive	0.19342	$\mathbf{6.699 \times 10^{-10}}$	0.28676	$\mathbf{2.589 \times 10^{-10}}$	0.110312	**0.007**
inc. fitness	inc. novelty	-0.114747	$\mathbf{1.036 \times 10^{-4}}$	-	0.411	-0.226368	$\mathbf{1.013 \times 10^{-4}}$
inc. fitness	fit. archive	-	0.982	0.134113	$\mathbf{2.369 \times 10^{-5}}$	-0.193233	$\mathbf{9.521 \times 10^{-8}}$
inc. fitness	elite archive	0.122055	$\mathbf{1.544 \times 10^{-4}}$	0.223159	$\mathbf{8.230 \times 10^{-10}}$	-	1.000
inc. fitness	novetly archive	0.125952	**0.047**	0.44541	$\mathbf{3.665 \times 10^{-10}}$	-0.18049	$\mathbf{9.294 \times 10^{-5}}$
inc. fitness	limit archive	-	0.950	0.28253	$\mathbf{6.380 \times 10^{-9}}$	-0.196636	$\mathbf{3.226 \times 10^{-5}}$
inc. novelty	fit. archive	0.111492	$\mathbf{9.680 \times 10^{-5}}$	0.173304	$\mathbf{9.250 \times 10^{-10}}$	-	1.000
inc. novelty	elite archive	0.243282	$\mathbf{3.123 \times 10^{-11}}$	0.259667	$\mathbf{3.665 \times 10^{-10}}$	0.222153	$\mathbf{5.161 \times 10^{-5}}$
inc. novelty	novetly archive	0.240971	$\mathbf{1.146 \times 10^{-7}}$	0.493238	$\mathbf{2.228 \times 10^{-11}}$	-	1.000
inc. novelty	limit archive	0.171545	$\mathbf{3.194 \times 10^{-5}}$	0.320165	$\mathbf{6.380 \times 10^{-9}}$	-	1.000
fit. archive	elite archive	0.137525	$\mathbf{1.653 \times 10^{-5}}$	-	0.182	0.188703	$\mathbf{1.043 \times 10^{-5}}$
fit. archive	novetly archive	0.141326	$\mathbf{1.912 \times 10^{-4}}$	0.287077	$\mathbf{7.464 \times 10^{-6}}$	-	1.000
fit. archive	limit archive	-	0.169	0.130784	**0.044**	-	1.000
elite archive	novetly archive	-	1.000	0.212562	**0.001**	-0.176027	$\mathbf{6.507 \times 10^{-4}}$
elite archive	limit archive	-	0.160	-	0.739	-0.181613	$\mathbf{1.100 \times 10^{-4}}$
novetly archive	limit archive	-	0.125	-0.133493	**0.015**	-	1.000

Table 5.24: Diversity Wilcoxon test over objectives handlers for CartPole .(HL_x - direction on algorithm x, p_x - p-value on algorithm x)

A	B	HL_{Total}	p_{Total}	HL_{DE}	p_{DE}	HL_{ES}	p_{ES}
fitness	pure novelty	-0.108583	$\mathbf{8.532 \times 10^{-14}}$	-0.06247	$\mathbf{3.308 \times 10^{-10}}$	-0.182832	$\mathbf{3.595 \times 10^{-10}}$
fitness	inc. fitness	-0.065009	$\mathbf{2.023 \times 10^{-11}}$	-0.063492	$\mathbf{2.700 \times 10^{-11}}$	-0.072573	$\mathbf{1.552 \times 10^{-4}}$
fitness	inc. novelty	-0.078057	$\mathbf{7.540 \times 10^{-12}}$	-0.064042	$\mathbf{1.869 \times 10^{-10}}$	-0.11046	$\mathbf{2.725 \times 10^{-5}}$
fitness	fit. archive	-	0.109	0.040538	**0.014**	-	1.000
fitness	elite archive	0.049615	**0.015**	0.133014	$\mathbf{4.640 \times 10^{-10}}$	-	0.071
fitness	novetly archive	-0.084113	$\mathbf{4.679 \times 10^{-5}}$	-	1.000	-0.19034	$\mathbf{8.441 \times 10^{-11}}$
fitness	limit archive	-	0.585	0.052496	**0.002**	-	0.442
pure novelty	inc. fitness	0.037016	**0.018**	-	1.000	0.105617	$\mathbf{4.264 \times 10^{-4}}$
pure novelty	inc. novelty	-	0.298	-	1.000	-	0.096
pure novelty	fit. archive	0.142925	$\mathbf{1.829 \times 10^{-13}}$	0.102334	$\mathbf{2.102 \times 10^{-8}}$	0.188583	$\mathbf{7.008 \times 10^{-10}}$
pure novelty	elite archive	0.175722	$\mathbf{1.346 \times 10^{-12}}$	0.192629	$\mathbf{1.592 \times 10^{-12}}$	0.144022	$\mathbf{4.114 \times 10^{-5}}$
pure novelty	novetly archive	0.043506	**0.018**	0.08317	$\mathbf{1.792 \times 10^{-6}}$	-	1.000
pure novelty	limit archive	0.13087	$\mathbf{6.004 \times 10^{-14}}$	0.110398	$\mathbf{1.215 \times 10^{-9}}$	0.156843	$\mathbf{1.301 \times 10^{-10}}$
inc. fitness	inc. novelty	-	0.585	-	1.000	-	0.442
inc. fitness	fit. archive	0.090774	$\mathbf{1.923 \times 10^{-11}}$	0.10164	$\mathbf{3.470 \times 10^{-9}}$	0.080122	$\mathbf{4.999 \times 10^{-5}}$
inc. fitness	elite archive	0.120897	$\mathbf{3.586 \times 10^{-9}}$	0.195151	$\mathbf{2.956 \times 10^{-12}}$	-	0.153
inc. fitness	novetly archive	-	0.940	0.087288	$\mathbf{4.616 \times 10^{-6}}$	-0.10732	$\mathbf{1.423 \times 10^{-5}}$
inc. fitness	limit archive	0.085673	$\mathbf{1.578 \times 10^{-8}}$	0.11279	$\mathbf{1.165 \times 10^{-8}}$	0.056607	**0.017**
inc. novelty	fit. archive	0.109141	$\mathbf{7.540 \times 10^{-12}}$	0.105241	$\mathbf{7.640 \times 10^{-10}}$	0.11848	$\mathbf{2.093 \times 10^{-5}}$
inc. novelty	elite archive	0.140465	$\mathbf{4.134 \times 10^{-11}}$	0.194627	$\mathbf{1.592 \times 10^{-12}}$	0.072567	**0.011**
inc. novelty	novetly archive	-	0.940	0.085309	$\mathbf{2.281 \times 10^{-7}}$	-0.06723	**0.011**
inc. novelty	limit archive	0.107013	$\mathbf{4.437 \times 10^{-9}}$	0.118202	$\mathbf{2.539 \times 10^{-8}}$	0.093966	**0.003**
fit. archive	elite archive	-	0.585	0.092345	$\mathbf{2.514 \times 10^{-5}}$	-0.035036	**0.008**
fit. archive	novetly archive	-0.107116	$\mathbf{3.948 \times 10^{-8}}$	-	1.000	-0.190565	$\mathbf{7.958 \times 10^{-12}}$
fit. archive	limit archive	-	0.940	-	1.000	-	0.104
elite archive	novetly archive	-0.126178	$\mathbf{2.357 \times 10^{-11}}$	-0.109312	$\mathbf{9.317 \times 10^{-7}}$	-0.144965	$\mathbf{2.440 \times 10^{-7}}$
elite archive	limit archive	-	0.298	-0.078102	$\mathbf{2.093 \times 10^{-4}}$	-	0.442
novetly archive	limit archive	0.093258	$\mathbf{1.459 \times 10^{-7}}$	-	1.000	0.16651	$\mathbf{4.188 \times 10^{-10}}$

Table 5.25: Diversity Wilcoxon test over objectives handlers for Lunar Lander. (HL_x - direction on algorithm x, p_x - p-value on algorithm x)

Conclusion

We have successfully developed and implemented framework for testing evolutionary strategies and differential evolution in several setups, such as fitness, novelty, their respective archiving variants and their combinations. We have also proposed behavioural characterisations for both environments Lunar Lander and CartPole. We have used the characteristics to investigate performance of objective handling methods derived from fitness, novelty and their combinations. We used the same series of experiments to determine a baseline hyper-parameter setting for each combination of environment, EA and objective handling method. On these setting we have performed statistical tests to determine relationships between the objective handling methods and their groupings based on the main objective used (fitness, novelty or combination). The areas of comparisons were: absolute performance, rate of convergence and diversity of the final populations.

We have identified that in the selected environments the strongest performance was obtained using the fitness based methods followed by the combinations with novelty. Novelty based methods were the weakest performing. We link this outcome to the complexity of selected environments. The CartPole environment is so simple that we obtain the solving individual through fitness only, the novelty can only provide more diverse populations. While Lunar Lander clearly has false optima the fitness based methods fall into, only a small strain of behaviours can reach it or surpass it. The novelty search attempting to explore the whole behavioural space is therefore rendered inefficient. We have also confirmed that dynamics of the search are very important. The elitist version of novelty archive with static threshold could generally not outperform *random novelty archive* in clear contrast to the *fitness archives* while either dynamic combinations of novelty were generally able to outperform both *novelty archives*. The results suggest that approach combining novelty and fitness or at least introducing controlled diversity into the population is therefore necessary to reliably obtain full solution. This confirms assumptions in literature delineating the usefulness of novelty such as Lehman and Stanley [2011] or Cuccu and Gomez [2011].

Main contribution of this thesis is the empirical evaluation of novelty-based objective handling methods in evolutionary reinforcement learning. The work provides a systematic comparison of fitness-based selection, novelty search, their archival variants, and a dynamic combination of fitness and novelty on two low deception benchmark environments. In addition, the thesis analyses the impact of these methods on optimisation performance and population diversity. It also proposes baseline behavioural characteristics for the selected environments and discusses the effects of the design on the analysis.

Our analysis was limited by the choice of behavioural characteristics being driven mostly by interpretability and not raw performance. Specifically for Lunar Lander it is possible that a more performance focused choice, perhaps utilising automated extraction of behavioural characteristics [and Gomes et al., 2014] would provide a clearer picture. However, optimisation of behavioural space was out of the scope of this thesis. Another limitation is the choice of evaluation scope for the Lunar Lander where we limited the number of steps and the action space.

In continuous version of Lunar Lander the agents might have been able to fines the last stage of landing more appropriately even without the use of novelty. Similar argument can be about expanding length of the episode. Loosening of these limitations however has tangible impact on effective performance of the evolutionary search as a whole. We have also chose to limit dynamics of the evolutionary strategy algorithm.

Several directions for future work emerge from this thesis. First, the experiments could be extended to a broader range of reinforcement learning environments to determine how broadly can our findings be generalised. This includes investigation of multiagent environments where the reward is based on behaviour of more than one agent such as the environment Waterworld from the Petting Zoo library [Terry et al., 2021]. Second, an investigation and cross analysis of the automatic algorithms for behavioural space extraction methods for the environments. Last but not least, a broader selection of evolutionary algorithms could be investigated as well, with focus on dynamic properties of the algorithms.

Bibliography

Takuya Akiba, Shotaro Sano, Takeru Yanase, Tadahiro Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.

Jorge and Gomes, Pedro Mariano, and Anders Christensen. Systematic derivation of behaviour characterisations in evolutionary robotics. In *Artificial Life 14: Proceedings of the Fourteenth International Conference on the Synthesis and Simulation of Living Systems*, ALIFE, page 212–219. The MIT Press, 2014. doi: 10.7551/978-0-262-32621-6-ch036. URL `http://dx.doi.org/10.7551/978-0-262-32621-6-ch036`.

Arthur Aubret, Laetitia Matignon, and Salima Hassas. An information-theoretic perspective on intrinsic motivation in reinforcement learning: A survey. *Entropy*, 25(2):327, February 2023. ISSN 1099-4300. doi: 10.3390/e25020327. URL `http://dx.doi.org/10.3390/e25020327`.

Ryan Bahlous-Boldi, Maxence Faldor, Luca Grillotti, Hannah Janmohamed, Lisa Coiffard, Lee Spector, and Antoine Cully. Dominated novelty search: Rethinking local competition in quality-diversity, 2025. URL `https://arxiv.org/abs/2502.00593`.

Vahid Beiranvand, Warren Hare, and Yves Lucet. Best practices for comparing optimization algorithms. *Optimization and Engineering*, 18(4):815–848, 2017. ISSN 1573-2924. doi: 10.1007/s11081-017-9366-1. URL `http://dx.doi.org/10.1007/s11081-017-9366-1`.

James S. Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2011.

Hans-Georg Beyer and Hans-Paul Schwefel. Evolution strategies - a comprehensive introduction. *Natural Computing*, 1:3–52, 03 2002. doi: 10.1023/A:1015059928466.

Giuseppe Cuccu and Faustino Gomez. When novelty is not enough. volume 6624, pages 234–243, 04 2011. ISBN 978-3-642-20524-8. doi: 10.1007/978-3-642-20525-5_24.

Edward E. Cureton. The normal approximation to the signed-rank sampling distribution when zero differences are present. *Journal of the American Statistical Association*, 62(319):1068–1069, 1967. doi: 10.1080/01621459.1967.10500917. URL `https://www.tandfonline.com/doi/abs/10.1080/01621459.1967.10500917`.

Stephane Doncieux, Alban Laflaquière, and Alexandre Coninx. Novelty search: a Theoretical Perspective. In *GECCO '19: Genetic and Evolutionary Computation Conference*, pages 99–106, Prague Czech Republic, France, July

2019. ACM. doi: 10.1145/3321707.3321752. URL https://hal.science/hal-02561846.

A. Eiben and Jim Smith. *Introduction To Evolutionary Computing*, volume 45. 01 2003. ISBN 978-3-642-07285-7. doi: 10.1007/978-3-662-05094-1.

Salvador García, Daniel Molina, Manuel Lozano, and Francisco Herrera. A study on the use of non-parametric tests for analyzing the evolutionary algorithms' behaviour: A case study on the cec'2005 special session on real parameter optimization. *J. Heuristics*, 15:617–644, 12 2009. doi: 10.1007/s10732-008-9080-4.

David E. Goldberg and Kalyanmoy Deb. A comparative analysis of selection schemes used in genetic algorithms. volume 1 of *Foundations of Genetic Algorithms*, pages 69–93. Elsevier, 1991. doi: https://doi.org/10.1016/B978-0-08-050684-5.50008-2. URL https://www.sciencedirect.com/science/article/pii/B9780080506845500082.

Jorge Gomes, Pedro Mariano, and Anders Lyhne Christensen. Devising effective novelty search algorithms: A comprehensive empirical study. In *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation*, GECCO '15, page 943–950, New York, NY, USA, 2015. Association for Computing Machinery. ISBN 9781450334723. doi: 10.1145/2739480.2754736. URL https://doi.org/10.1145/2739480.2754736.

J. L. Hodges Jr. and E. L. Lehmann. Estimates of Location Based on Rank Tests. *The Annals of Mathematical Statistics*, 34(2):598 – 611, 1963. doi: 10.1214/aoms/1177704172. URL https://doi.org/10.1214/aoms/1177704172.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1):99–134, 1998. ISSN 0004-3702. doi: https://doi.org/10.1016/S0004-3702(98)00023-X. URL https://www.sciencedirect.com/science/article/pii/S000437029800023X.

Joel Lehman and Kenneth Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary computation*, 19:189–223, 06 2011. doi: 10.1162/EVCO_a_00025.

Elliot Meyerson, Joel Lehman, and Risto Miikkulainen. Learning behavior characterizations for novelty search. In *Proceedings of the Genetic and Evolutionary Computation Conference 2016*, GECCO '16, page 149–156, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450342063. doi: 10.1145/2908812.2908929. URL https://doi.org/10.1145/2908812.2908929.

Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites, 2015. URL https://arxiv.org/abs/1504.04909.

Nemanja Rakicevic, Antoine Cully, and Petar Kormushev. Policy manifold search for improving diversity-based neuroevolution. *CoRR*, abs/2012.08676, 2020. URL https://arxiv.org/abs/2012.08676.

Stéphane Ross, Joelle Pineau, Sébastien Paquet, and Brahim Chaib-draa. Online planning algorithms for pomdps. *CoRR*, abs/1401.3436, 2014. URL http://arxiv.org/abs/1401.3436.

Stuart Russell and Peter Norvig. *Artificial Intelligence, Global Edition A Modern Approach*. Pearson Deutschland, 2021. ISBN 9781292401133. URL https://elibrary.pearson.de/book/99.150005/9781292401171.

Achkan Salehi, Alexandre Coninx, and Stephane Doncieux. Br-ns: an archiveless approach to novelty search. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '21, page 172–179. ACM, 2021. doi: 10.1145/3449639.3459303. URL http://dx.doi.org/10.1145/3449639.3459303.

Neil Spencer, Nigel Smeeton, and Peter Sprent. *Applied Nonparametric Statistical Methods*. 03 2025. ISBN 9780429326172. doi: 10.1201/9780429326172.

Rainer Storn and Kenneth Price. Differential evolution - a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11:341–359, 01 1997. doi: 10.1023/A:1008202821328.

J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Rodrigo Perez, Caroline Horsch, Clemens Dieffendahl, Niall L. Williams, Yashas Lokesh, and Praveen Ravi. Pettingzoo: Gym for multi-agent reinforcement learning, 2021. URL https://arxiv.org/abs/2009.14471.

Frank Wilcoxon. *Individual Comparisons by Ranking Methods*, pages 196–202. Springer New York, New York, NY, 1992. ISBN 978-1-4612-4380-9. doi: 10.1007/978-1-4612-4380-9_16. URL https://doi.org/10.1007/978-1-4612-4380-9_16.

List of Figures

List of Tables

List of Abbreviations

EA	Evolutionary Algorithm
ES	Evolutionary Strategy
DE	Differential Evolution
BS	Bayesian Search
IGS	Incremental grid search

A. Attachments

A.1 Full grid search charts

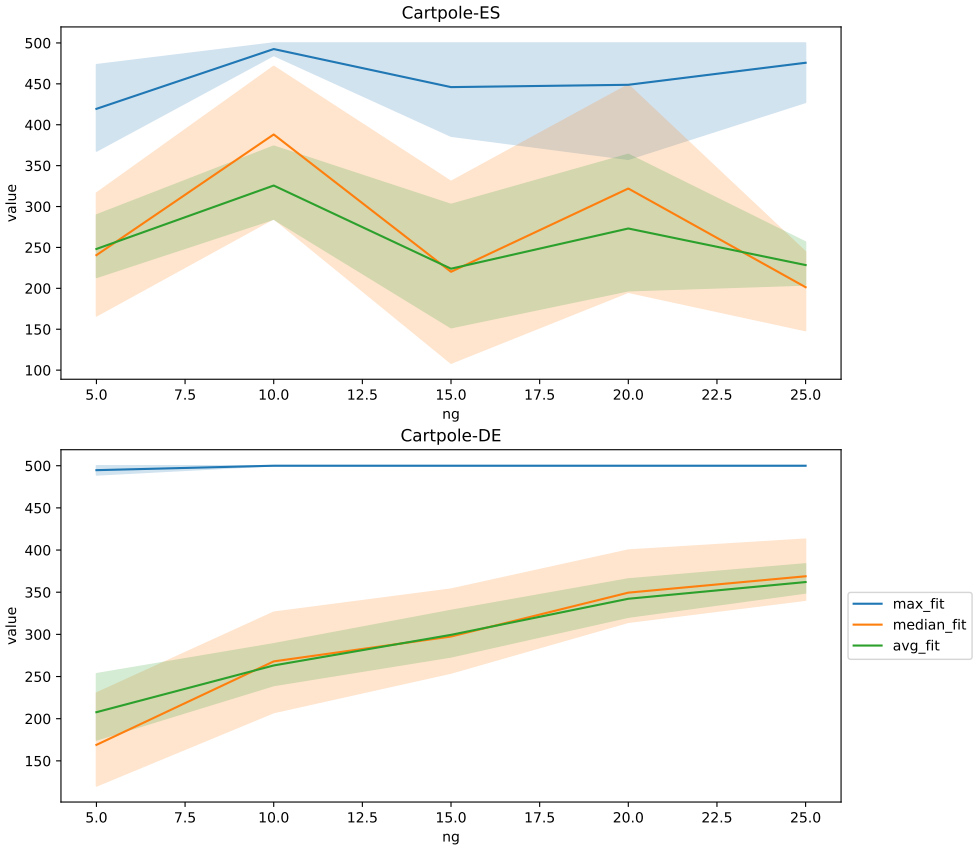

Figure A.1: Detailed fitness/generation characteristic for fitness on Cartpole env.

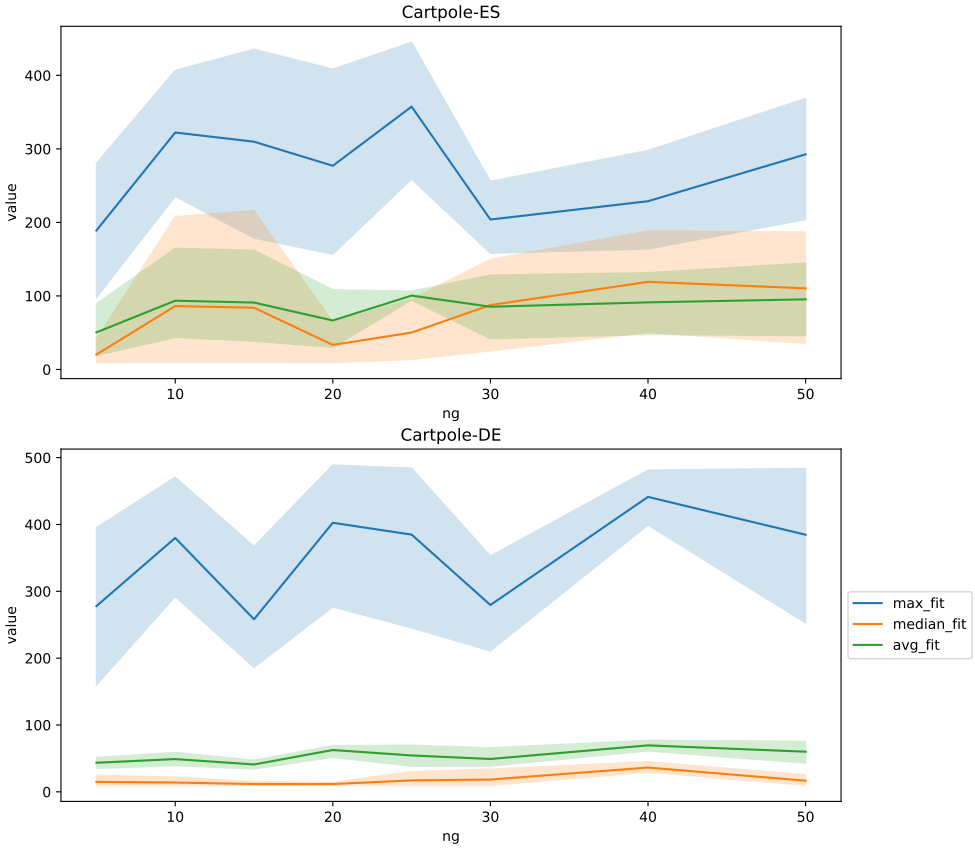

Figure A.2: Detailed fitness/generation characteristic for pure novelty on Cartpole env.

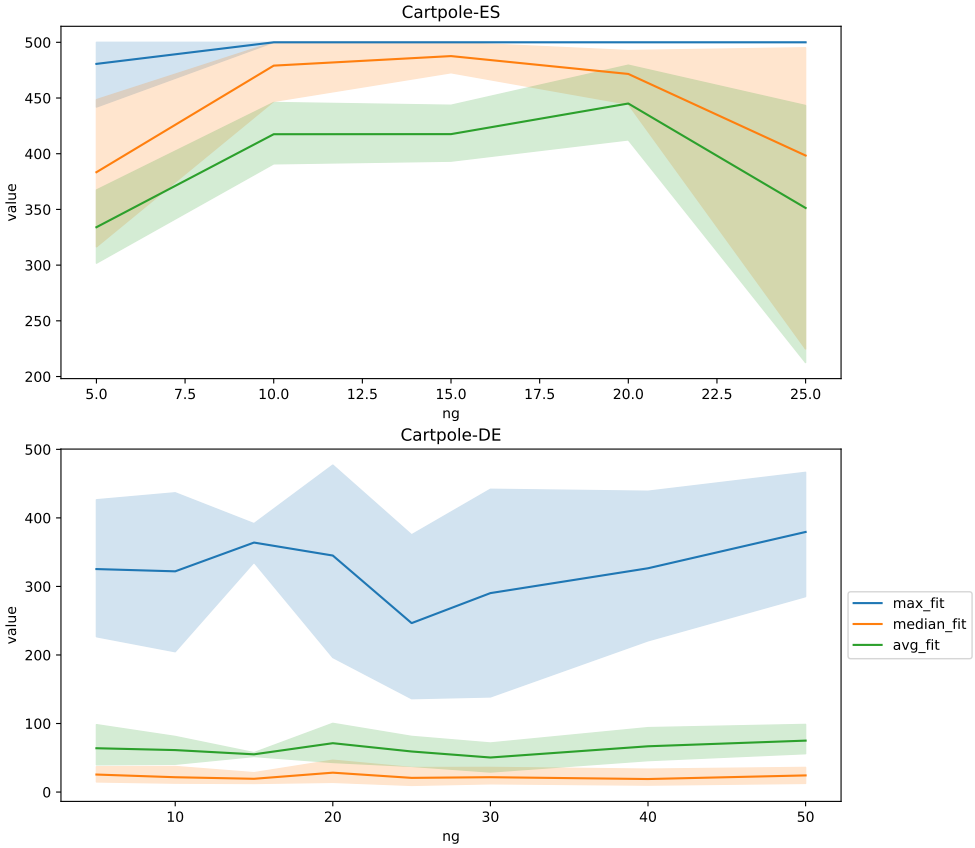

Figure A.3: Detailed fitness/generation characteristic for inc. fitness on Cartpole env.

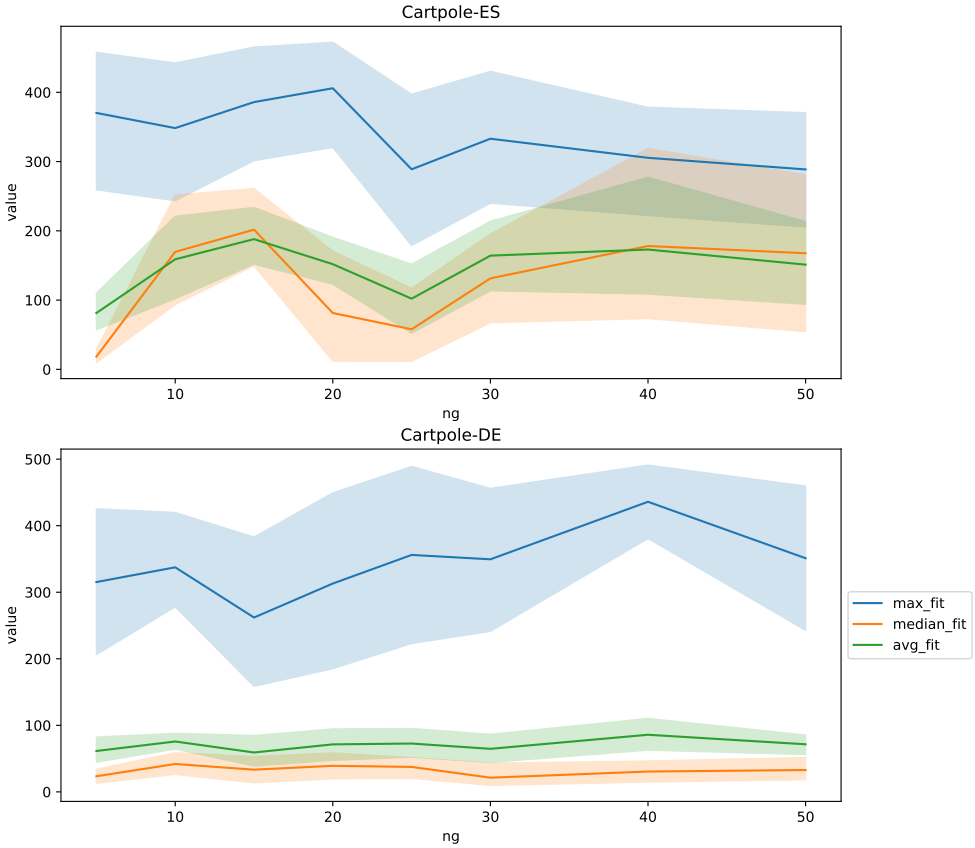

Figure A.4: Detailed fitness/generation characteristic for inc. novelty on Cartpole env.

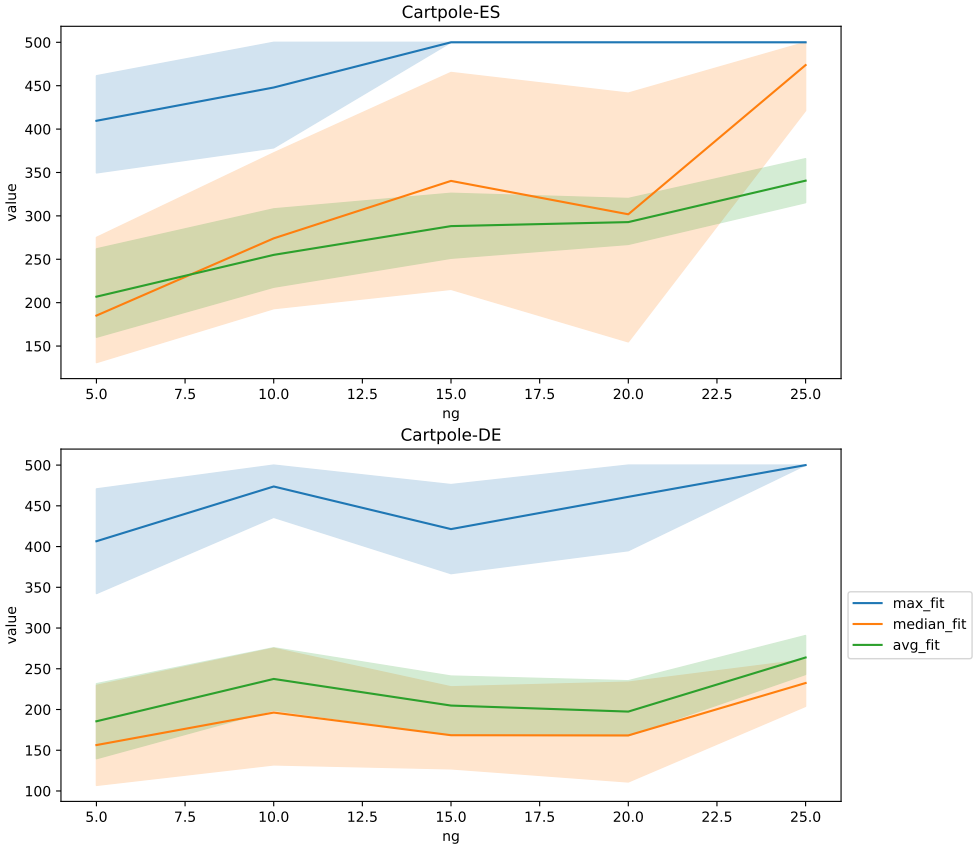

Figure A.5: Detailed fitness/generation characteristic for fit. archive on Cartpole env.

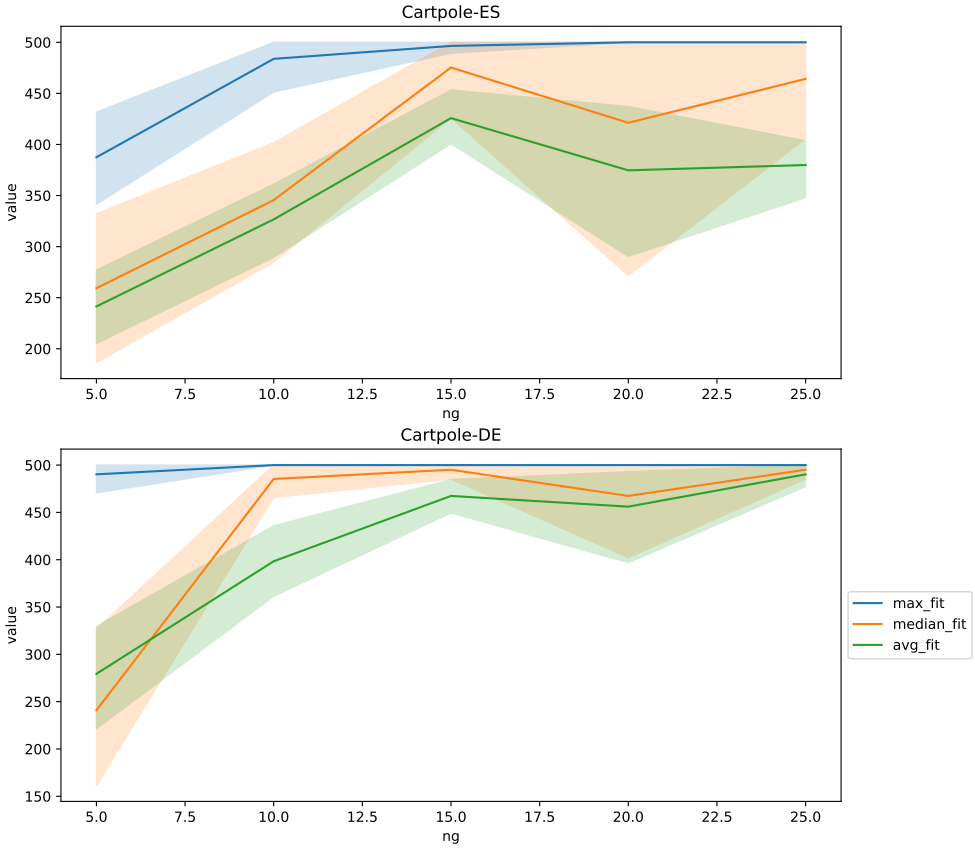

Figure A.6: Detailed fitness/generation characteristic for elite archive on Cartpole env.

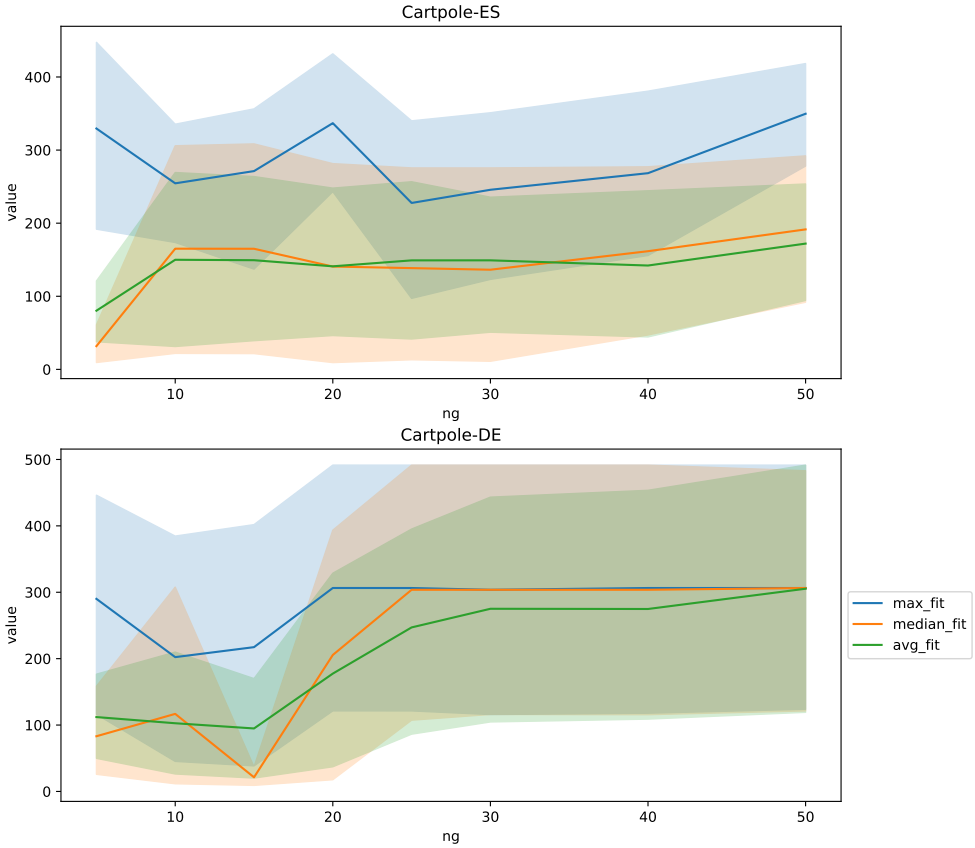

Figure A.7: Detailed fitness/generation characteristic for novetly archive on Cartpole env.

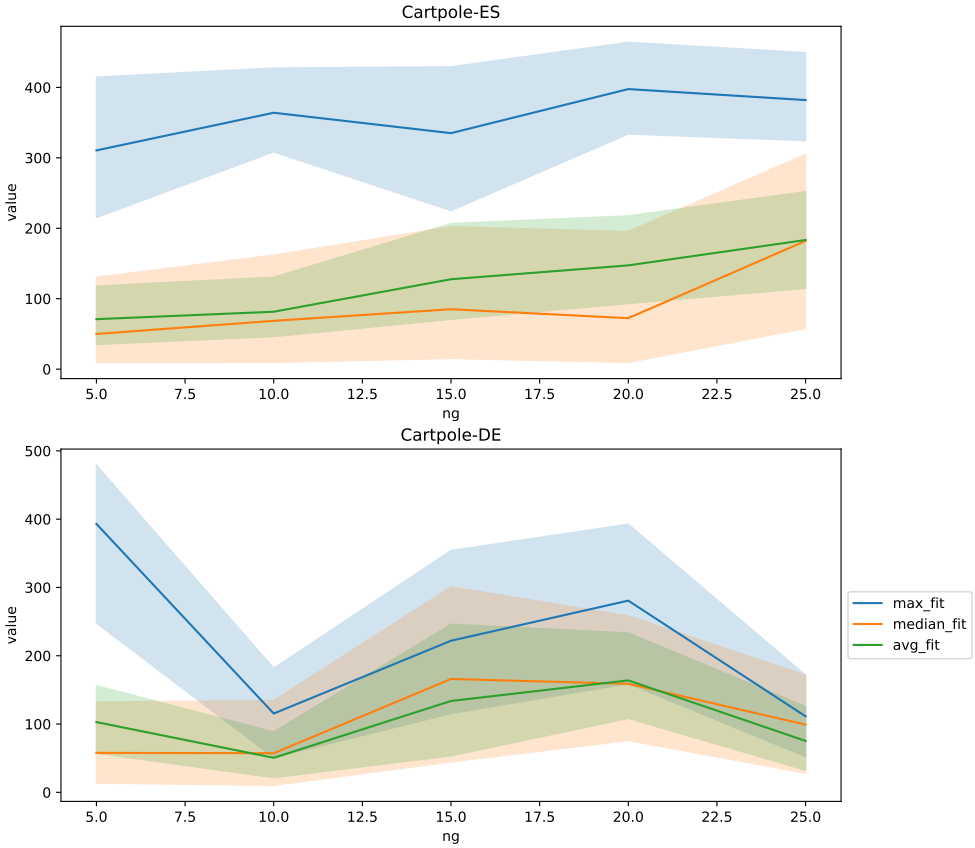

Figure A.8: Detailed fitness/generation characteristic for limit archive on Cartpole env.

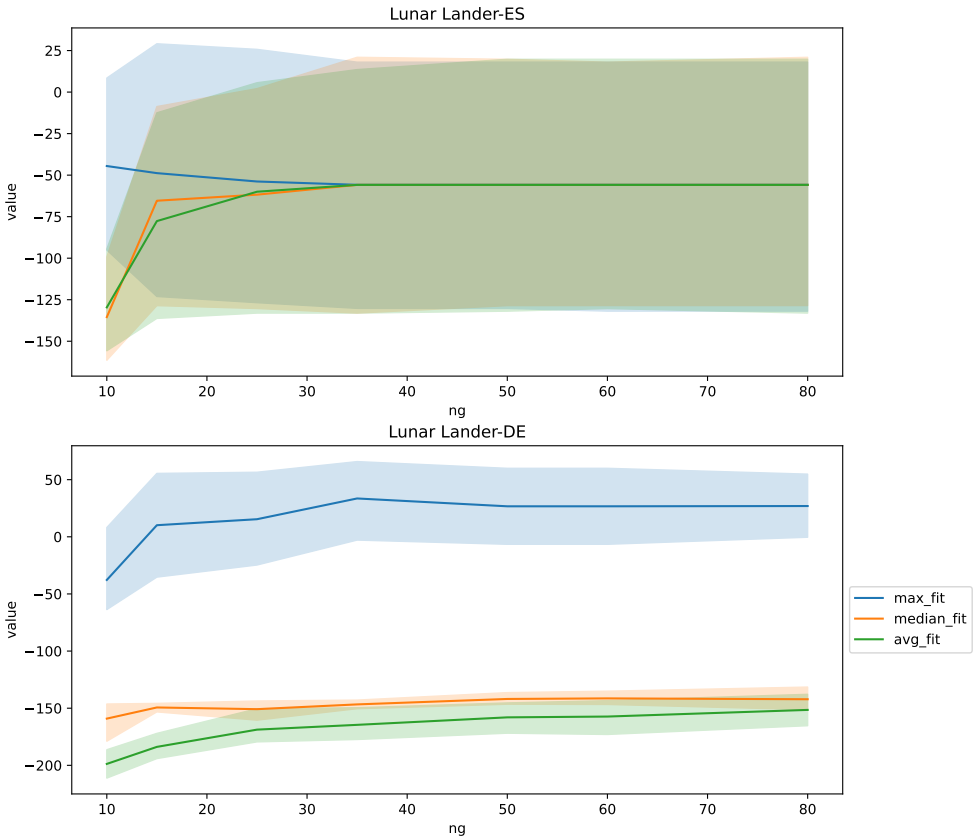

Figure A.9: Detailed fitness/generation characteristic for fitness on Lunar Lander env.

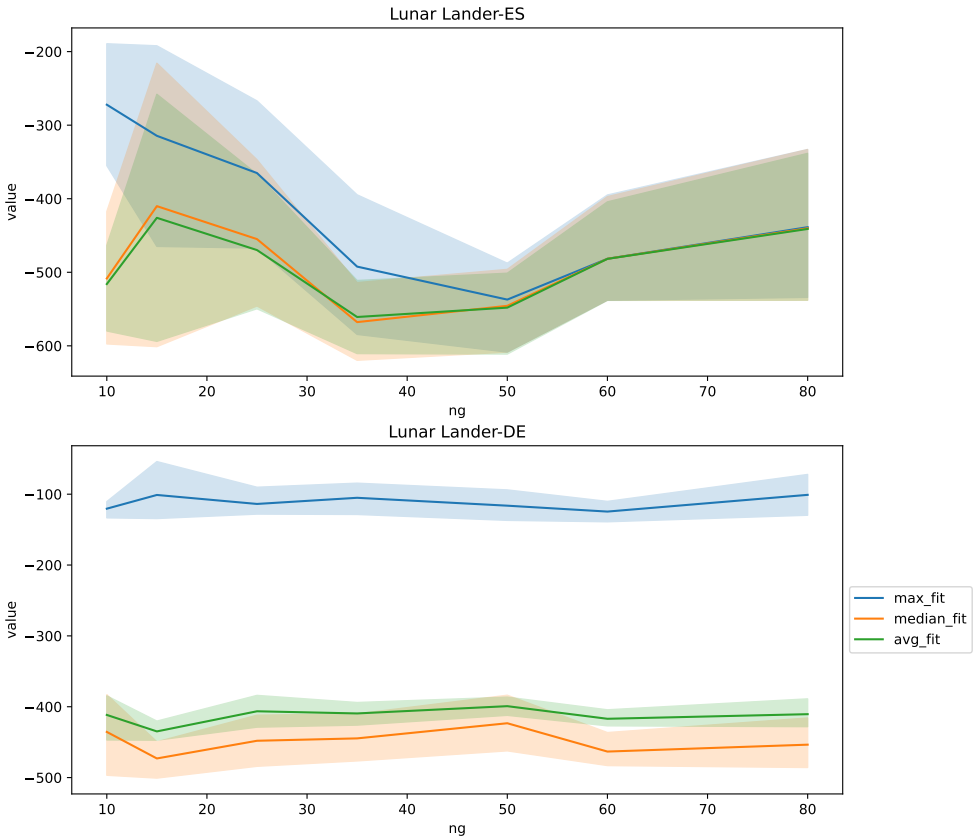

Figure A.10: Detailed fitness/generation characteristic for pure novelty on Lunar Lander env.

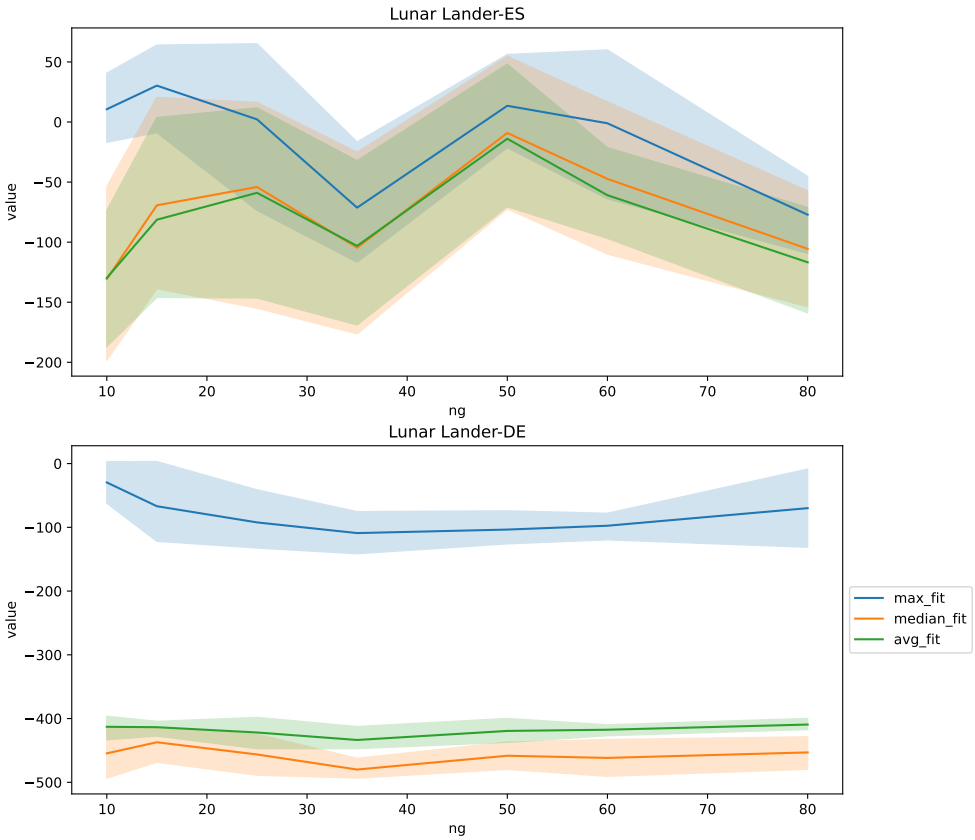

Figure A.11: Detailed fitness/generation characteristic for inc. fitness on Lunar Lander env.

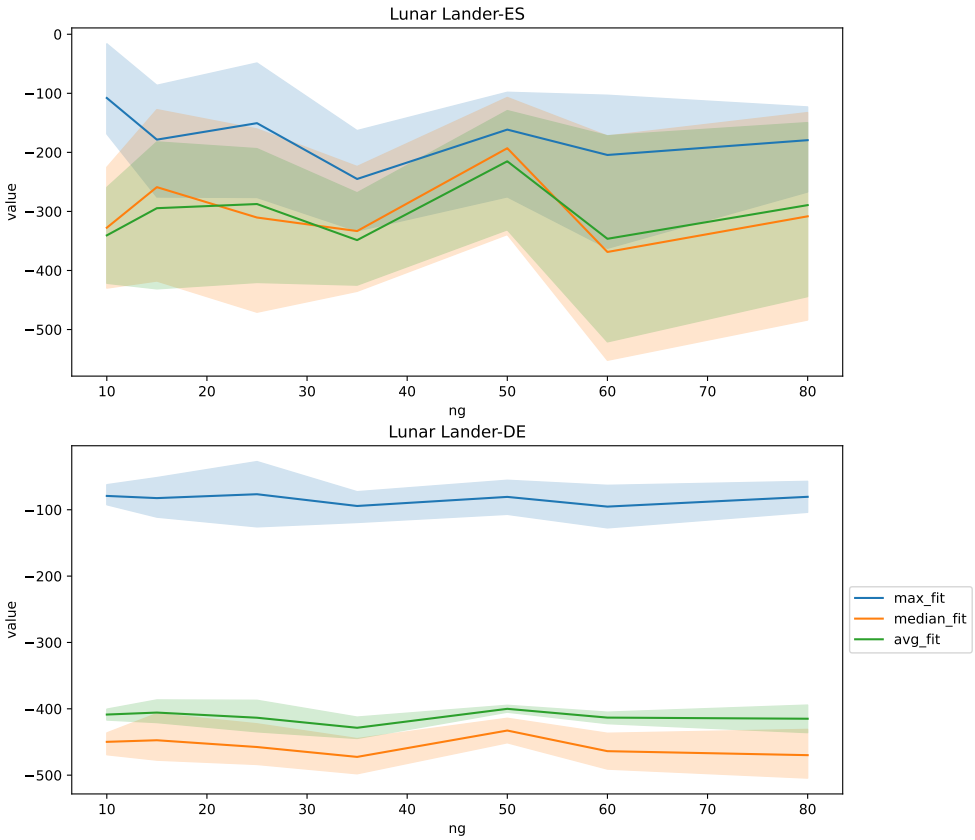

Figure A.12: Detailed fitness/generation characteristic for inc. novelty on Lunar Lander env.

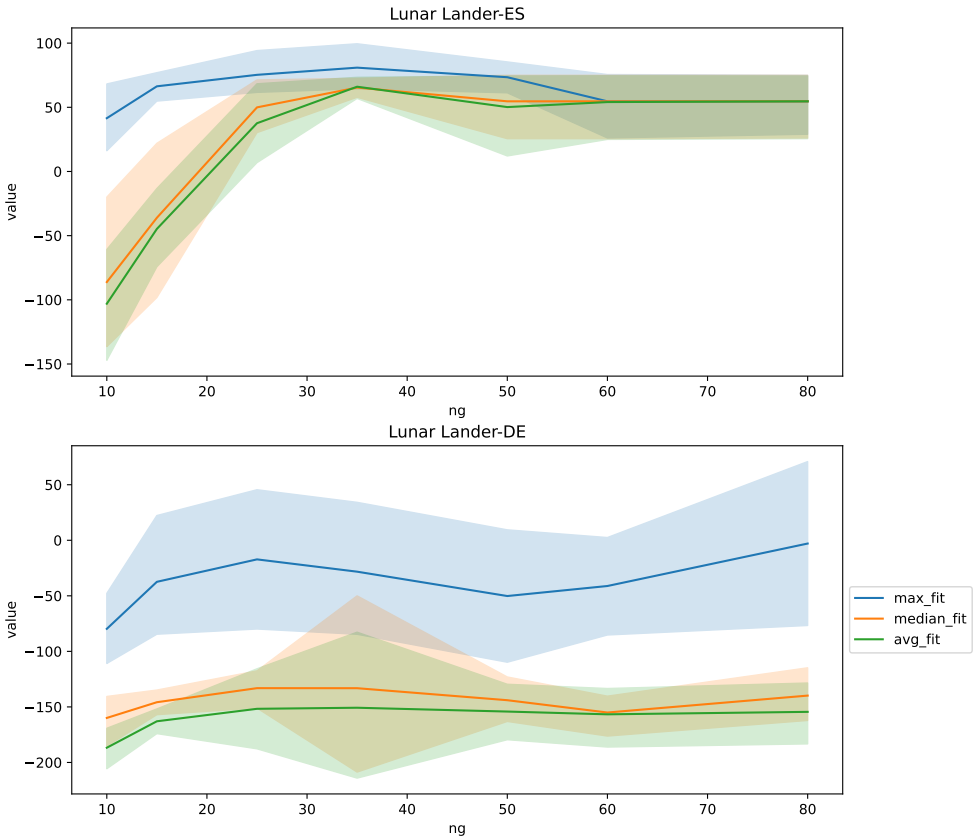

Figure A.13: Detailed fitness/generation characteristic for fit. archive on Lunar Lander env.

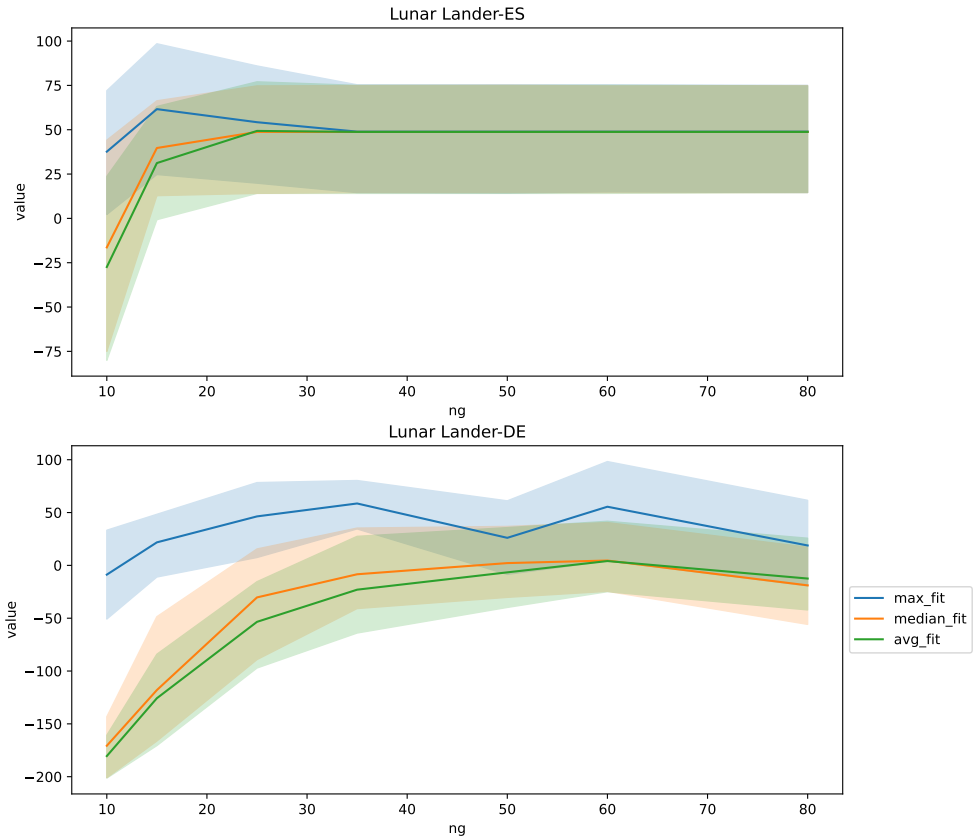

Figure A.14: Detailed fitness/generation characteristic for elite archive on Lunar Lander env.

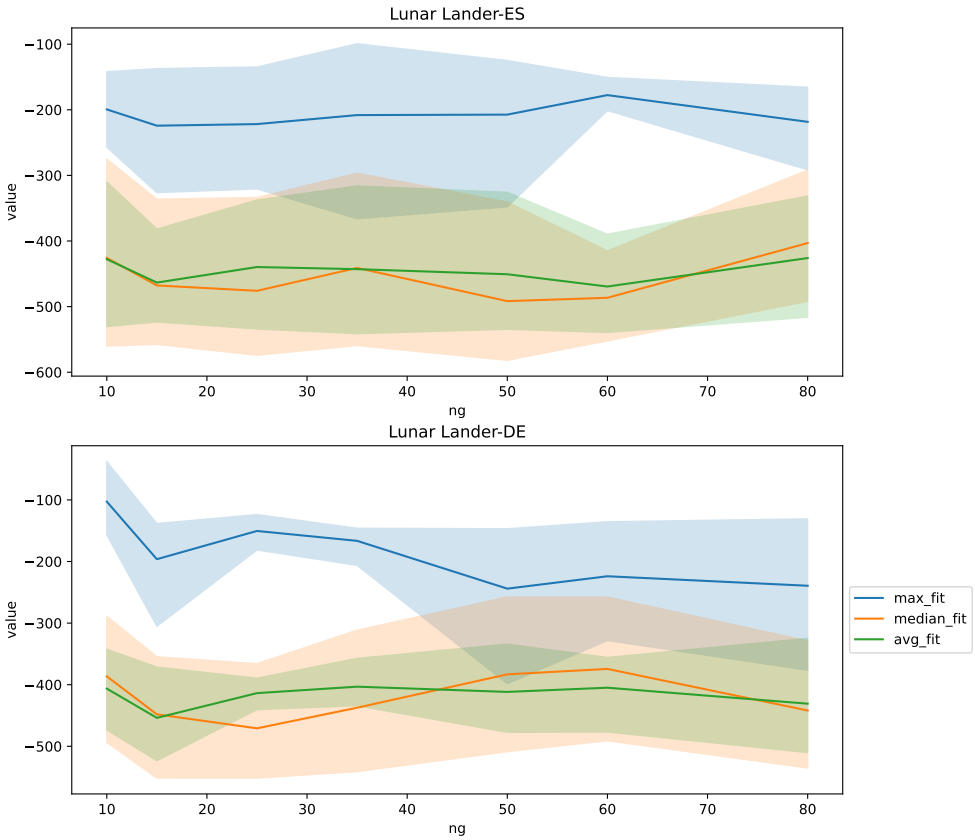

Figure A.15: Detailed fitness/generation characteristic for novetly archive on Lunar Lander env.

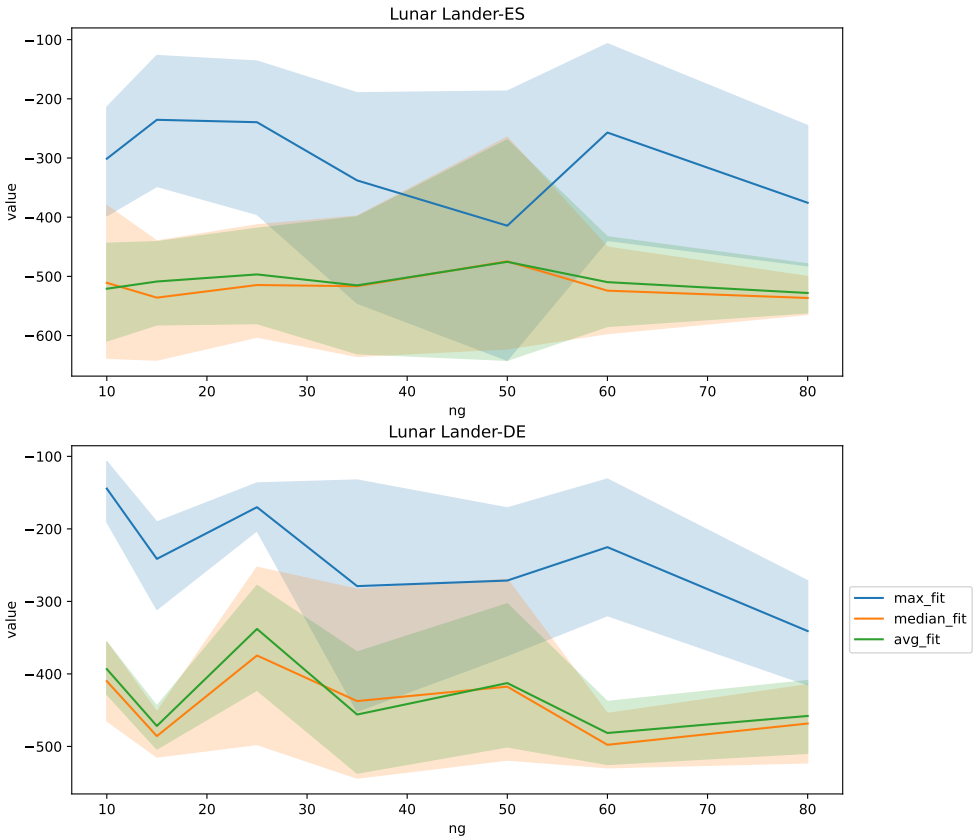

Figure A.16: Detailed fitness/generation characteristic for limit archive on Lunar Lander env.

A.2 Project structure

Project

- `Archive/` Archived experiments and outputs (unfeatured)
- `Data/` Datasets and generated data
- `Docs/` Thesis LaTeX files
- `libs/` Implementations of the algorithms
- `notebooks/` Helper Jupyter notebooks
- `scripts/` Automation Bash scripts
- `adaptive_gridsearch.py` Incremental grid search
- `diff_cont.py` DE evaluation
- `final_examination.py` Final evaluation
- `generation_examination.py` Generation evaluation
- `lambda_cont.py` ES evaluation
- `optuna_experiment.py` Optuna experiments
- `view_episodes.py` Replay
- `visualisation.py` Visualisation utilities
- `README.md` Project documentation

Figure A.17: Project directory structure.

A.3 Experiment Data Format

A.3.1 Incremental Grid Search Protocol

```
{
   "name": <experiment name>,
   "start": <start timestamp>,
   "end": <end timestamp>,

   "origin": [
      <list of hyperparameter sets>
   ],

   "final": <Final hyperparameter set>,

   "visited": [
      <list of hyperparameter sets>
   ]
}
```

A.3.2 Generations Grid Protocol

```
{
   "fitness": {
      <# of generations>:{

         <seed no.>:[
            [
               [<fitness>],
               [<behavioral_vector>]
            ],
            ...,
            [
               [<fitness>],
               [<behavioral_vector>]
            ]
         ],
         ...
      },
      ...
   }
   "environment":<environment id>,
   "algorithm":<algorithm id>,
   "container"<container id>,
   "arguments":{
      <hyperparameter id>: <hyperparameter value>,
      ...
   }
}
```

A.3.3 Final Experiment Protocol

```
{
   "fitness": {
      <seed no.>:[
         [
            [<fitness>],
            [<behavioral_vector>]
         ],
         ...
      ],
      ...
   }
   "environment":<environment id>,
   "algorithm":<algorithm id>,
   "container"<container id>,
   "arguments":{
      <hyperparameter id>: <hyperparameter value>,
```

```
        ...
    }
}
```

A.4 Used Hardware

Lenovo ThinkPad E16 Gen 3
Lenovo Legion 5
HP ProLiant DL320e Gen8 v2